

TipJar – Platforma Mikropłatności Web3 dla Twórców Treści

Wprowadzenie i Koncept

TipJar to nowatorska platforma mikropłatności (napiwków) oparta o technologię Web3, która umożliwia fanom wspieranie internetowych twórców treści (youtuberów, streamerów, blogerów itp.) drobnymi kwotami pieniężnymi w postaci stablecoina USDC. Głównym celem projektu jest zminimalizowanie barier i kosztów przy przekazywaniu nawet symbolicznych kwot (np. \$1–5), eliminując wysokie prowizje znane z tradycyjnych platform typu Patreon czy YouTube (sięgające ~30%). Dzięki TipJar twórca w kilka minut zakłada profil i otrzymuje unikalny link oraz kod QR, za pośrednictwem których fani mogą przekazywać napiwki – globalnie i praktycznie bezkosztowo.

Kluczowe cechy koncepcji:

Wykorzystanie stablecoina USDC: Stablecoin USDC doskonale nadaje się do mikrotransakcji globalnych. Zapewnia stabilną wartość (1 USDC $\approx$ 1 USD) i może być przesyłany niemal bez opłat na tanich sieciach blockchain (np. Polygon, Arbitrum). Dzięki temu napiwek \$1 wysłany z Azji ma tę samą wartość dla twórcy w Europie, a przy tym żadna ze stron nie martwi się o zmienność kursu krypto. USDC eliminuje też problemy przewalutowań – twórcy zyskują natychmiastowy dostęp do środków w „silnej” walucie cyfrowej.

Minimalne prowizje dla platformy: TipJar może utrzymywać się z niewielkiej prowizji (np. 1–2%) od transakcji lub ewentualnych opcji premium, co w porównaniu z ~30% na tradycyjnych platformach jest ogromną różnicą. Twórca otrzymuje niemal całą wpłaconą kwotę, co zwiększa atrakcyjność korzystania z systemu.

Globalny zasięg, brak granic: Platforma działa w skali globalnej – fan z dowolnego kraju może wesprzeć twórcę z innego kraju w ciągu sekund. Brak jest ograniczeń geograficznych czy wysokich opłat międzynarodowych. TipJar omija tradycyjne systemy płatności, wykorzystując globalną sieć blockchain.

Szybki onboarding twórców: Rejestracja twórcy na TipJar jest uproszczona maksymalnie – możliwa przez konto społecznościowe (Google, Twitch, YouTube itp.) lub portfel kryptowalutowy (Sign-In with Ethereum). Od razu tworzony jest dla twórcy dedykowany portfel USDC (custodial, kontrolowany przez system TipJar poprzez API Circle), dzięki czemu twórca nie musi sam zakładać ani zarządzać własnym portfelem krypto. Proces startu ogranicza się do kilku kliknięć.

Intuicyjny UX dla fanów: Fan odwiedzający profil twórcy na TipJar widzi prosty formularz „Wesprzyj kwotą: [slider \$1–\$100]” oraz przycisk do dokonania płatności. Może zapłacić kryptowalutą (USDC) – np. poprzez integrację z własnym portfelem Web3 typu MetaMask – lub nawet kartą płatniczą, przy czym środki są automatycznie konwertowane na USDC w tle (dzięki integracji z partnerami fiat on-ramp, np. usługami Circle). Wszystko to odbywa się bez potrzeby zakładania konta przez fana w serwisie (choć TipJar może zachęcać do łatwego utworzenia portfela w systemie dla jeszcze prostszych płatności w przyszłości).

Proces przekazania napiwku jest maksymalnie uproszczony i szybki, bez odczuwalnych kosztów dodatkowych.

Natychmiastowa wypłacalność środków: Twórca może w dowolnym momencie wypłacić zgromadzone USDC – na swój własny portfel kryptowalutowy lub bezpośrednio na konto bankowe (poprzez partnerów obsługujących konwersję USDC→FIAT, np. rozwiązania Circle oferujące wypłaty na rachunki bankowe). Wypłata jednym kliknięciem powoduje, że TipJar zleca odpowiednią transakcję z portfela twórcy. Twórcy mogą też trzymać otrzymane USDC, by np. wykorzystać je w ekosystemie DeFi między wypłatami.

Brak zmartwień o “gas” (opłaty transakcyjne): TipJar korzysta z mechanizmów Circle Gas Station / Paymaster, dzięki którym fani wysyłający napiwki nie muszą płacić za gas na blockchainie. Przy mikropłatnościach nawet drobna opłata (np. \$0,10) mogłaby zniechęcić do wysłania \$1 napiwku. Platforma TipJar automatycznie sponsoruje te opłaty sieciowe na zapleczu – zwłaszcza że operuje na tanich sieciach (Polygon, Solana itp.), więc koszty są minimalne. W razie potrzeby TipJar może rozliczać gas w USDC (np. poprzez mechanizm “Pay User’s Gas in USDC” oferowany przez Circle). To zapewnia płynne doświadczenie użytkownika: fan płaci dokładnie \$1 i nie obchodzi go techniczna warstwa opłat sieciowych.

Rozwiązywany problem: Monetyzacja drobnych treści i napiwków dla twórców od lat nasyca trudności. Istniejące platformy:

Są często ograniczone geograficznie (np. Patreon nie obsługuje wielu krajów) i generują wysokie koszty przewalutowań oraz wypłat międzynarodowych.

Pobierają wysokie prowizje, przez co z małego napiwku niewiele zostaje dla twórcy.

Mają bariery minimalnej kwoty lub opłaty transakcyjne (np. przy płatności kartą małe kwoty są nieopłacalne ze względu na prowizje płatnicze).

Nie wykorzystują potencjału kryptowalut/stablecoinów, które mogłyby te problemy rozwiązać.

TipJar adresuje te problemy, oferując:

Globalny, walutowo neutralny system napiwków – stablecoiny działają 24/7 w skali świata.

Znikome koszty transakcyjne – dzięki blockchainom L2 i stablecoinom.

Brak progów wejścia – fani mogą wesprzeć nawet symbolicznie, twórcy otrzymują niemal całość kwoty.

Nowe możliwości monetyzacji i interakcji – integracja z transmisjami na żywo (alerty napiwków), możliwość ustawiania celów finansowych, subskrypcji itp.

Promowanie adopcji Web3 – twórcy i ich społeczności poznają praktyczne zastosowanie kryptowalut (stablecoinów) w sposób przynoszący realną wartość.

Specyfikacja Funkcjonalna

Funkcje dla Twórców (Creator Features)

Zakładanie profilu twórcy: Twórca może zarejestrować się błyskawicznie, używając istniejącego konta (Google, Twitch, YouTube, TikTok itp.) lub opcji Web3 (Sign-In with Ethereum – MetaMask). Rejestracja tworzy unikalny profil pod adresem `tipjar.com/@nazwa` oraz generuje odpowiadający mu kod QR. Podczas rejestracji w tle zakładany jest powiązany portfel USDC dla twórcy (custodial, oparty o Circle API).

Personalizacja profilu: Twórca ma możliwość uzupełnienia swojego profilu o zdjęcie/awatar, baner graficzny, opis bio, a także ustalenia domyślnego komunikatu powitalnego dla fanów. Może również określić cele zbiorów (np. „Zbieram 1000 USDC na nowy obiekt”) – wówczas na profilu wyświetlany jest pasek postępu osiągnięcia celu.

Widget i link do napiwków: Każdy twórca może wygenerować kod do osadzenia widgetu TipJar na własnej stronie WWW lub otrzymuje gotowy link, który może umieścić w opisie filmu/streamu. Widget wyświetla podstawowe informacje (nazwa twórcy, opis, cel) oraz przycisk „Wesprzyj [X] USDC”.

Panel twórcy (dashboard): Po zalogowaniu twórca ma dostęp do panelu administracyjnego, gdzie widzi:

Saldo portfela (bieżąca ilość USDC dostępna).

Historię otrzymanych napiwków – lista transakcji z informacją o kwocie, dacie, nadawcy (jeśli nieanonimowy) oraz opcjonalną wiadomością od fana.

Statystyki i analizy – np. łączna suma wsparcia, miesięczne podsumowania, liczba unikalnych fanów wspierających itp.

Opcje wypłaty środków: Twórca może zlecić wypłatę wybranej kwoty ze swojego portfela TipJar:

Na zewnętrzny adres portfela kryptowalutowego (transfer on-chain USDC).

Na konto bankowe (sprzedaż USDC i przelew fiat – wymaga integracji z bramką fiat, np. Circle Payouts).

Ustawienia konta: edycja profilu, ustawienia powiadomień (np. e-mail/SMS o otrzymaniu napiwku), konfiguracja preferencji (np. preferowana sieć blockchain do otrzymywania napiwków, waluta wyświetlania itp.).

Powiadomienia w czasie rzeczywistym: Po otrzymaniu napiwku twórca może dostać natychmiastowe powiadomienie (push w aplikacji mobilnej, e-mail, lub nawet powiadomienie na streamie poprzez integrację z OBS/Streamlabs). API TipJar może udostępniać webhook lub wtyczkę dla oprogramowania streamerskiego, aby na ekranie pojawiała się animacja „ $\$$ {fan} wsparł Cię kwotą 5 USDC!”.

Subskrypcje i wsparcie cykliczne: (Funkcjonalność planowana) Twórca może zaoferować fanom opcję miesięcznego wsparcia (np. 5 USDC co miesiąc). System TipJar będzie wówczas automatycznie inicjował cykliczny transfer USDC z portfela fana do portfela twórcy (wymaga to albo autoryzacji wielokrotnej transakcji w Circle API, albo wykorzystania smart kontraktu/subskrypcji on-chain).

Weryfikacja tożsamości (opcjonalna): Twórcy mogą zweryfikować swoje konto (np. poprzez połączenie z kontami społecznościowymi lub dostarczenie dodatkowych danych) aby uzyskać odznakę „Zweryfikowany Twórca”. To buduje zaufanie wśród fanów, że środki trafiają do właściwej osoby. (Od strony regulacyjnej, TipJar na starcie nie wymaga pełnego KYC, ponieważ operuje drobnymi kwotami; jednak w razie dużych przepływów lub wypłat fiat, platforma może wymagać od twórców przejścia procedury KYC zgodnie z przepisami AML.)

Funkcje dla Fanów (Fan Features)

Przekazywanie napiwków (tipowanie): Fan odwiedzający stronę twórcy (lub używający widgetu TipJar osadzonego np. na blogu twórcy) widzi prosty interfejs wsparcia:

Wybór kwoty napiwku (pole lub suwak z przedziałem, np. od 1 USDC do 100 USDC, z możliwością ręcznego wpisania innej kwoty).

Pole opcjonalnej wiadomości dla twórcy (np. „Dziękuję za świetny stream!”).

Wybór metody płatności:

Krypto (USDC): Fan może zapłacić korzystając z portfela kryptowalutowego. Jeśli posiada MetaMask (lub inny Web3 wallet) – pojawia się przycisk „Zapłać krypto”, który wywołuje okno potwierdzenia transakcji. System TipJar udostępnia adres portfela twórcy (w odpowiedniej sieci, np. Polygon) i kwotę do wysłania – po zatwierdzeniu w MetaMask środki są przesyłane on-chain. Tip: TipJar może monitorować blockchain w poszukiwaniu tej transakcji (np. nasłuchując eventów transferu USDC na adres twórcy) i po potwierdzeniu, zaksięgować napiwek w systemie.

Portfel TipJar (Circle): Jeśli fan założy prosty portfel w TipJar (np. logując się również przez Google i automatycznie dostając własny portfel USDC), wówczas przekazanie napiwku sprowadza się do wewnętrznego transferu między dwoma portfelami w systemie (fan → twórca) – to może być jedno kliknięcie. Uwaga: W początkowej fazie zakładamy, że fan raczej nie ma swojego portfela w systemie, więc ta opcja jest bardziej perspektywiczna.

Karta płatnicza (fiat): Dla fanów nieobeznanych z kryptowalutami TipJar może oferować bramkę płatniczą – np. integrację z Circle Payments API lub innym dostawcą – aby zapłacić zwykłą kartą (Visa/Mastercard). W tym scenariuszu fan wprowadza dane karty jak w klasycznym checkoucie, następuje autoryzacja np. 3D Secure, a kwota np. \$5 zostaje pobrana i automatycznie zamieniona na 5 USDC, które trafia do portfela twórcy. Dla fana jest to transparentne – po prostu zapłacił kartą na rzecz twórcy. (Takie rozwiązanie wymaga pewnych procedur zgodności regulacyjnej, ale Circle i podobne firmy oferują gotowe rozwiązania on-ramp.)

Tryb anonimowy vs. podpisany: Fan może wybrać, czy chce przedstawić się twórcy (np. imieniem lub nickiem) i przekazać wiadomość, czy woli pozostać anonimowy. Jeśli fan jest zalogowany/posiada profil TipJar, system może podpisać jego napiwek automatycznie nazwą profilu.

Potwierdzenie i podziękowanie: Po zrealizowaniu transakcji fan otrzymuje wizualne potwierdzenie (animacja „Dziękujemy za wsparcie!”). W przypadku integracji na żywo, twórca mógł otrzymać alert na ekranie streamu. Fanom można wyświetlić dowód transakcji (np. skrót TX na blockchainie dla przejrzystości, w razie płatności krypto).

Profil fana (opcjonalnie): Jeśli fan utworzy konto (opcjonalne, np. by mieć historię swoich napiwków czy zbierać odznaki), może mieć prosty profil z podstawowymi danymi, listą twórców których wsparł, itp. Jednak TipJar nie wymaga od fanów zakładania konta – „pay as guest” jest domyślnym, by usunąć tarcie.

Gamifikacja dla fanów: (Opisano szczegółowo w sekcji Elementy Gamifikacji) Fani mogą zdobywać pewne odznaki lub wyróżnienia za aktywność (np. „Super Fan” – za wsparcie powyżej określonej kwoty, leaderboard top wspierających danego twórcę, itp.), co zachęca do udziału w zabawie i buduje społeczność.

Funkcje Administracyjne i Infrastrukturalne

Moderacja i support: Zespół TipJar (administratorzy) posiada panel administracyjny do przeglądu i moderacji treści profili (np. czy opisy nie naruszają regulaminu), rozpatrywania zgłoszeń nadużyć, pomocy użytkownikom w przypadku problemów (np. zagubione dostępy). Admini mogą w razie potrzeby wstrzymać wypłatę lub zamrozić konto twórcy, jeśli wykryto podejrzane działania (fraudy, pranie pieniędzy).

Konfiguracja opłat i prowizji: System pozwala ustawić globalną prowizję platformy (np. 1% od napiwku) lub zróżnicować ją w zależności od wielkości twórcy/pakietu (np. plan premium bez prowizji w zamian za abonament). Takie reguły są zaimplementowane w backendzie – np. przy każdym napiwku X% jest przekazywane na konto/platformy lub odkładane do osobnego portfela prowizyjnego.

Monitorowanie transakcji: TipJar integruje monitoring przepływów – zarówno on-chain (nasłuchiwanie kontraktu USDC dla adresów portfeli twórców), jak i off-chain (logi operacji w

systemie). Pozwala to wykrywać ewentualne problemy (np. nieudane transakcje, duże opóźnienia w finalizacji) i generować alerty.

Skalowalność i rozszerzenia: Architektura jest projektowana modularnie, co umożliwia dodawanie kolejnych funkcjonalności:

Wsparcie kolejnych sieci blockchain dla USDC (np. Avalanche, Solana) – dzięki Circle CCTP możliwe staje się przekazywanie USDC między sieciami, jeśli np. fan ma środki na Ethereum, a twórca preferuje otrzymywać na Polygon, system automatycznie to obsłuży (mostkowanie stablecoina podczas transferu).

Wprowadzenie innych tokenów w przyszłości (np. napiwki w lokalnych stablecoinach lub innych walutach cyfrowych) – obecnie jednak fokus jest na USDC jako najbardziej uniwersalnym.

Integracje społecznościowe – np. TipJar mógłby oferować plugin dla Discorda czy Twittera, pozwalający błyskawicznie wysłać napiwek poprzez komendę lub kliknięcie w mediach społecznościowych.

Architektura Systemu

Architektura TipJar opiera się na modelu klient-serwer z wydzielonym zapleczem integrującym usługi blockchain poprzez API firmy Circle. W skład systemu wchodzi następujące główne komponenty:

Front-end TipJar: Aplikacje klienckie, z którymi interakcję mają użytkownicy (twórcy i fani). Obejmują one aplikację webową (dashboard twórców, strony profilowe, landing page platformy) stworzoną w technologii React/Next.js, a także planowaną aplikację mobilną (prawdopodobnie w React Native lub Flutter) dla twórców. Front-end zawiera również łatwy do osadzenia widget webowy (np. fragment HTML/JS) do umieszczenia na stronach twórców.

Backend TipJar (API): Wyspecjalizowany serwer aplikacyjny zbudowany w Node.js z użyciem frameworka NestJS. Zapewnia on logikę biznesową i udostępnia API (REST/HTTP, ewentualnie WebSockets do powiadomień) dla front-endu oraz integruje się z usługami zewnętrznymi:

Baza danych (np. PostgreSQL poprzez ORM Prisma) do przechowywania danych o użytkownikach, profilach, transakcjach, itp.

Circle API – do zarządzania portfelami USDC i realizacji transferów.

Zewnętrzne API OAuth – do obsługi logowania przez platformy społecznościowe (Google, Twitch, itp.).

Inne usługi – np. system wysyłki e-maili z potwierdzeniami, notyfikacje push, usługa analityki itp.

Platforma Circle (infrastruktura zewnętrzna): Obejmuje Circle Programmable Wallets (programowalne portfele custodial), Circle Gas Station/Paymaster (usługa sponsorowania gazu transakcyjnego) oraz Circle CCTP (Cross-Chain Transfer Protocol do przenoszenia USDC między sieciami). TipJar backend komunikuje się z tymi usługami przez oficjalne SDK/REST API. Ponadto wykorzystywani mogą być partnerzy Circle do obsługi płatności kartą i wypłat fiat (Circle Payments/Payouts).

Sieć blockchain: Warstwa blockchain (początkowo jedna lub dwie sieci, np. Polygon jako główna sieć do transferów USDC, opcjonalnie Ethereum/Avalanche do kompatybilności z innymi ekosystemami). Blockchain zapewnia właściwą realizację transferów wartości (USDC) między adresami. Dzięki integracji Circle, wiele operacji (jak transfer między portfelami twórców) może być obsługiwanych off-chain w ramach infrastruktury Circle i jej rozrachunku on-chain w razie potrzeby.

Usługi wspomagające i narzędzia DevOps: Kontenery Docker do uruchamiania komponentów, pipeline CI/CD do automatycznego testowania i wdrażania, serwery hostingowe (np. AWS, Azure lub inne) obsługujące backend, monitoring i logowanie (np. Sentry do błędów, Prometheus/Grafana do metryk wydajności).

Poniższy diagram przedstawia kluczowe elementy architektury i ich zależności (komponenty TipJar w niebieskich ramkach, usługi zewnętrzne na zielono, użytkownicy na szaro, strzałki obrazują przepływ danych i wartości):

Użytkownik (Fan) [przeglądarka / mobilnie, MetaMask lub karta]

|

v

****TipJar Frontend**** (Aplikacja Web / Widget / Mobile)

| REST API (żądania HTTP)

v

****TipJar Backend**** (NestJS API + baza danych)

| integracja z Circle

v

****Circle Platform**** (Wallets API, Gas Station, CCTP)

| transakcje USDC on-chain

v

****Sieć Blockchain**** (np. Polygon, Ethereum)

|

v

****Portfel Twórcy (Circle)**** – środki USDC dostępne

|

v

Użytkownik (Twórca) [panel TipJar, wypłaty]

(Diagram tekstowy: Fan przesyła żądanie wsparcia poprzez frontend → backend TipJar tworzy/zleca transakcję w Circle → Circle wykonuje operację w sieci blockchain (jeśli potrzebna) → środki trafiają do portfela twórcy w systemie. Twórca widzi to w swoim panelu i może wypłacić dalej.)

Architektura została zaprojektowana tak, aby bezpieczeństwo, skalowalność i niezawodność stały na wysokim poziomie:

Bezpieczeństwo warstwy blockchain i portfeli: Portfele twórców korzystają z technologii MPC (Multi-Party Computation) oferowanej przez Circle – klucze prywatne są dzielone i bezpiecznie przechowywane, co minimalizuje ryzyko kompromitacji. Dodatkowo wszystkie operacje na portfelach są logowane audytowo (kto, kiedy, jaka operacja) zarówno w systemie TipJar, jak i w logach blockchain (hash transakcji).

Bezpieczeństwo aplikacji i danych: Backend stosuje najlepsze praktyki bezpieczeństwa Node.js/NestJS – walidacja danych wejściowych (pakiet class-validator zabezpieczający przed wstrzyknięciami), mechanizmy rate-limiting (nestjs-throttler ogranicza np. liczbę prób logowania na IP), użycie WAF (np. Cloudflare) przed najczęstszymi atakami webowymi. Dane w bazie są szyfrowane „w spoczynku” (np. pole circleWalletId czy inne wrażliwe identyfikatory mogą być dodatkowo szyfrowane alg. AES-256). Sekrety API (klucze OAuth, klucze Circle) są przechowywane wyłącznie w zmiennych środowiskowych/menedżerach sekretów – nigdy w kodzie źródłowym.

Warstwa autoryzacji i kontroli dostępu: Całe API TipJar wymaga autoryzacji JWT dla operacji chronionych (panel twórcy itp.). Dostępów administracyjnych są zabezpieczone dodatkowo (np. 2FA dla adminów). Mechanizm OAuth używany do logowania twórców ogranicza scope danych pobieranych z ich kont (minimum potrzebne, np. e-mail i ID). Dla Sign-In with Ethereum generowane nonces zapobiegają atakom powtórzeniowym, a podpisy są weryfikowane po stronie backendu przy użyciu biblioteki kryptograficznej (ethers.js).

Skalowalność i wydajność: System od początku przewiduje możliwość poziomego skalowania:

Backend napisany w NestJS jest bezstanowy (stateless) – dzięki wykorzystaniu JWT i bazie danych do utrwalania stanu, można uruchamiać wiele instancji API za load balancerem.

Wykorzystanie asynchroniczności Node.js oraz mechanizmu kolejek zadań (np. BullMQ + Redis) pozwala przenosić cięższe operacje (jak tworzenie portfela w Circle czy wysyłanie e-maili) do przetwarzania w tle, nie blokując odpowiedzi HTTP. Np. po rejestracji twórcy można asynchronicznie utworzyć portfel Circle – twórca natychmiast dostaje odpowiedź o założeniu konta, a portfel dołącza po chwili (co sygnalizuje mu UI).

Użycie tanich i szybkich sieci L2 (Polygon) zapewnia, że nawet duża liczba małych transakcji nie zatyka sieci ani nie generuje opóźnień. Gas Station sponsoruje opłaty, a TipJar może w razie wzrostu skali zaopatrywać się w zasoby gazu hurtowo (np. trzymać pewną pulę MATIC do opłacania transakcji).

Baza danych (PostgreSQL) może być skalowana (np. replikacja odczytów). Prisma jako ORM zapewnia optymalizacje zapytań, a kluczowe kolumny są indeksowane (np. email, googleId, circleWalletId itp. mają indeksy).

Monitoring wydajności (APM) – zintegrowano narzędzia typu Datadog/New Relic, które mierzą czasy odpowiedzi API, czas realizacji zewnętrznych wywołań (np. do Circle czy Google). Pozwala to wykryć wąskie gardła i zaplanować optymalizacje.

Niezawodność i obsługa błędów: Każde wywołanie zewnętrzne (API Google, API Circle itd.) ma zaimplementowane mechanizmy ponawiania (retry z eksponencjalnym odstępem) w razie chwilowych błędów sieci. Wykorzystujemy np. bibliotekę axios z ustawionym mechanizmem axios-retry dla zapytań HTTP. Ponadto, jeśli pewne operacje nie powiodą się wiele razy, trafiają do Dead Letter Queue (kolejki błędów) – co wyzwala alert do obsługi technicznej. System loguje błędy (np. do Sentry), dzięki czemu zespół od razu wie o wyjątku w produkcji.

Idempotentność jest zapewniona tam, gdzie to potrzebne – np. podczas tworzenia portfela w Circle używany jest unikalny idempotencyKey (UUID), by powtórne wywołanie nie tworzyło duplikatów. Circle API wspiera idempotentność i TipJar to wykorzystuje.

Kompatybilność i przyszłościowość: Zastosowane technologie są nowoczesne, ale i sprawdzone. Backend NestJS łatwo integruje kolejne strategie logowania czy kolejne usługi (modułowa architektura). Warstwa integracji z Circle jest odseparowana w formie np. CircleService – dzięki czemu, gdyby kiedykolwiek zaszła potrzeba zmiany dostawcy lub przejścia na bardziej zdecentralizowane rozwiązanie (np. ERC-4337 Account Abstraction z własnym smart kontraktem portfela), będzie to możliwe do zaadaptowania bez przebudowy całego systemu.

Komponenty Backendowe

Backend TipJar zbudowany w NestJS został podzielony na moduły odpowiadające poszczególnym domenom logiki. Poniżej opis najważniejszych komponentów i usług backendu:

Moduł Auth – odpowiada za uwierzytelnianie twórców (oraz ewentualnie fanów, jeśli zakładają konta). Zawiera:

Strategie OAuth2 dla logowania zewnętrznego: np. GoogleStrategy, TwitchStrategy itp. (wykorzystujące bibliotekę Passport.js i jej strategie OAuth). Każda strategia po pomyślnym zalogowaniu zwraca podstawowe dane profilu (ID zewnętrzne, email, avatar).

Strategia SIWE (Sign-In with Ethereum): obsługuje proces logowania portfelem kryptowalutowym. Implementuje m.in. endpointy /auth/siwe/nonce (generowanie nonce i przechowanie go tymczasowo) oraz /auth/siwe/verify (weryfikacja podpisu użytkownika, odzyskanie adresu i zalogowanie).

AuthController z endpointami: /auth/google (redirect na Google OAuth), /auth/google/callback (odbiór danych od Google i wydanie JWT), analogiczne dla innych dostawców oraz np. /auth/siwe/verify. Po zalogowaniu generowany jest token JWT (biblioteka @nestjs/jwt), który jest zwracany klientowi (np. ustawiany jako cookie HttpOnly lub zwracany w body), by kolejne żądania mogły uwierzytelniać użytkownika.

AuthService – zawiera logikę sprawdzania użytkowników w bazie, tworzenia nowych kont po raz pierwszy (np. nowy twórca loguje się przez Google – serwis sprawdza: czy mamy w bazie użytkownika z danym googleid lub emailiem? Jeśli nie – tworzy nowy rekord użytkownika, w tym inicjuje dla niego proces tworzenia portfela Circle).

Moduł Users/Profile – zarządza danymi profili twórców. Pozwala pobierać publiczne profile (na potrzeby stron twórców), edytować profil (prywatne API dla właściciela), ustawiać cele, itp. To tutaj następuje integracja z AuthService przy tworzeniu użytkownika, aby w tle uruchomić provisioning portfela (patrz integracja z Circle niżej).

Moduł Tips/Payments – obsługuje logikę przekazywania napiwków. Zapewnia endpointy:

Publiczny endpoint inicjujący płatność (np. /tips/initiate) wywoływany z frontendu kiedy fan chce zapłacić. W zależności od metody:

Jeśli fan płaci krypto bez integracji z TipJar (np. używa własnego MetaMask), endpoint może wygenerować unikalny adres (lub pobrać z bazy adres portfela twórcy) i kwotę, zwracając je do frontendu wraz z ewentualnym żądaniem podpisu/autoryzacji przez portfel. Dalej system czeka na potwierdzenie on-chain (np. poprzez usługę webhook/monitoring).

Jeśli fan płaci poprzez swój portfel TipJar (wewnętrzny transfer) – wtedy wywołanie wymaga autoryzacji (fan musi być zalogowany). Serwis weryfikuje saldo fana i zleca transfer wewnętrzny (patrz integracja Circle: Transfers).

Jeśli fan płaci kartą – serwis może integrować się z API bramki płatniczej. Np. wywołuje API Circle Payments tworząc tzw. PaymentIntent i przekazuje frontowi klienta token do obsługi (lub generuje link do kasy). Po zatwierdzeniu transakcji przez dostawcę płatności, otrzymuje webhook potwierdzający, który finalizuje płatność.

Endpointy webhooków/listenerów:

Webhook płatności fiat – odbiera sygnał od dostawcy (np. Circle), że płatność kartą się powiodła, wraz ze szczegółami (kwota, który fan/twórca, itp.), i następnie zasila portfel twórcy odpowiadającą ilością USDC.

Monitorowanie on-chain – tu są różne podejścia: albo nasłuchiwanie zdarzeń (subskrypcja logów kontraktu USDC dla adresów portfeli twórców), albo regularny polling przez Circle API (SDK Circle może udostępniać listę transakcji portfela). Gdy wykryty zostanie nowy depozyt od fana, serwis oznacza transakcję jako zakończoną i rejestruje napiwek.

PaymentsService – incydentalnie może korzystać z CircleService (poniżej) do inicjowania transferów między portfelami w systemie.

Moduł CircleIntegration (CircleService) – kapsuluje komunikację z API Circle. Inicjalizuje oficjalny klient SDK @circle-fin/developer-controlled-wallets. Odpowiada za:

Tworzenie portfeli twórców: metoda provisionUserWallet(userId, email) wywołuje Wallets API Circle tworząc nowy portfel w ramach wcześniej utworzonego Wallet Set TipJar (jest to wymaganie Circle – wszystkie portfele należą do tzw. entity TipJar i są pogrupowane). Metoda ustawia parametry:

idempotencyKey – unikalny UUID dla idempotentności żądania.

walletSetId – identyfikator zestawu portfeli (z konfiguracji, przypisany TipJar).

blockchains – lista sieci, np. ["POLYGON"] – wybór głównej sieci, na której portfel będzie aktywny.

accountType: "SCA" – typ konta smart (Sponsor Controlled Account) umożliwiający sponsorowanie gazu na EVM.

description/metadata – np. przypisanie userId, email dla identyfikacji.

Po otrzymaniu odpowiedzi, serwis zapisuje w bazie otrzymany circleWalletId (unikalny UUID portfela w systemie Circle) oraz adres USDC tego portfela (mainWalletAddress – to jest adres na blockchainie, np. rozpoczynający się od 0x na Polygon).

Operacja jest wykonywana asynchronicznie, aby nie opóźniać rejestracji użytkownika – np. za pomocą kolejki zadań: event "walletNeeded" trafia do BullMQ, a worker wywołuje CircleService.provisionUserWallet.

Transfery USDC (w ramach Circle): Jeśli zarówno fan jak i twórca mają portfele w systemie (developer-controlled wallets), możliwy jest transfer off-chain: Circle pozwala na przesunięcie środków między dwoma portfelami prowadzonymi przez tę samą entity. CircleService udostępnia metodę np. transferUSDC(fromWalletId, toWalletId, amount) która wywołuje Circle API (endpoint Transfers) z parametrami: portfel źródłowy, docelowy, kwota, idempotencyKey. Dzięki temu napiwek może zostać przekazany natychmiastowo w ramach systemu (Circle księguje to wewnętrznie, a ewentualny ruch on-chain jest przeźroczysty).

Wyплаты (on-chain lub fiat): CircleService może również inicjować wypłatę na zewnętrzny adres blockchain – to de facto transfer z portfela twórcy na adres wskazany (Circle wykona transakcję on-chain z użyciem Gas Station by pokryć opłatę). Alternatywnie, może wywołać API Circle Payouts żeby zlecić przelew bankowy (to już bardziej zaawansowane, wymagające skonfigurowania beneficjentów, KYC twórcy itp., więc najpewniej realizowane na dalszym etapie).

Obsługa błędów i wyjątków: CircleService centralizuje try/catch dla wywołań do Circle. W razie błędów (np. brak środków, błąd sieci, przekroczone limity) – loguje problem, ewentualnie rzuca wyjątek `HttpException` do wyższych warstw by zwrócić błąd API klientowi (np. 502 Bad Gateway jeśli problem z zewnętrzną usługą).

Moduł Notifications – wysyła powiadomienia o nowych napiwkach. Może integrować się z usługą e-mail (np. SendGrid) do powiadomienia twórcy „Otrzymałeś nowe wsparcie 5 USDC od użytkownika X”. Albo dla fanów – potwierdzenie e-mailem płatności. W przyszłości moduł ten rozbuduje się o webpush czy integracje z API YouTube/Twitch (by wyświetlać powiadomienie na streamie).

Moduł Analytics – rejestruje zdarzenia (eventy) do analizy zachowań: np. konwersje na stronie głównej (ile osób kliknęło „Wesprzyj”, ile sfinalizowało płatność), trendy wysokości napiwków itp. Pozwala to zespołowi TipJar ulepszać UX i też tworzyć raporty dla twórców.

Schemat bazy danych: Warto wspomnieć uproszczony schemat głównych tabel:

User – informacje o użytkowniku (twórcy, ewentualnie fanie jeśli rejestruje konto):

id (UUID, klucz główny),

email (unikalny),

googleId, twitchId, youtubeId... (opcjonalne identyfikatory powiązane z logowaniem OAuth, unikalne),

walletAddressSIWE (adres Ethereum, jeśli logował się przez portfel Web3),

circleWalletId (UUID portfela Circle powiązanego, jeśli utworzono),

mainWalletAddress (adres USDC na blockchain dla tego portfela),

username (unikalna nazwa użytkownika do profilu, np. @alias),

displayName, bio, avatarURL, bannerURL (dane profilu),

goalAmount, goalDescription (cel zbiórki jeśli ustawiony),

... (inne ustawienia, timestampy, itp.).

Tip – zapis pojedynczej transakcji napiwku:

id (UUID),

fromUserId (może być null jeśli anonim lub gość),

toUserId (twórca, FK do User),

amount (ilość USDC, decimal),

message (wiadomość od fana, jeśli załączył),

timestamp,

txHash (jeśli transakcja odbyła się on-chain, zapisujemy hash dla referencji),

paymentMethod (ENUM: USDC_INTERNAL, USDC_ONCHAIN, CARD, etc. – sposób przekazania),

status (INITIATED, PENDING, CONFIRMED, FAILED).

Ewentualnie tabele: Subscription, Withdrawal, OAuthTokens (jeśli przechowujemy refresh tokeny do API społecznościowych, choć to wrażliwe – można unikać i polegać na stateless JWT), AuditLog itp.

Komponenty Frontendowe

Warstwa frontendu TipJar obejmuje aplikację webową, widget oraz docelowo aplikację mobilną. Wszystkie one powinny zapewniać spójne i przyjazne doświadczenie użytkownika, zgodne z zaprojektowanym design systemem (o którym niżej). Technologicznie frontendy bazują na nowoczesnym stosie: React (z TypeScript), z wykorzystaniem frameworka Next.js dla strony web (co umożliwi SSR dla lepszej wydajności SEO strony głównej i profili) oraz potencjalnie React Native dla aplikacji mobilnej.

Główne elementy frontendu:

Strona główna platformy (Landing Page): Prezentuje kluczowe informacje o TipJar dla nowych użytkowników i partnerów. Zawiera hasło przewodnie, korzyści platformy, przykładowe statystyki, sekcję „Jak to działa?”, listę popularnych twórców oraz CTA do rejestracji. (Patrz sekcja Design System (UI/UX) – opisano tam stylistykę i układ).

Moduł rejestracji/logowania: Interfejs umożliwiający twórcom logowanie przez wybranego dostawcę (przyciski „Zaloguj przez Google/Twitch...” oraz opcja „Zaloguj portfelem Web3”). W przypadku OAuth – po kliknięciu następuje przekierowanie do zewnętrznej usługi autoryzacji (realizowane przez backend Endpoint /auth/...), a po powrocie front otrzymuje w przeglądarce ciasteczko JWT potwierdzające zalogowanie. Przy SIWE – wyświetlany jest modal z prośbą o podpisanie wiadomości w MetaMask, a wynik jest wysyłany do backendu do weryfikacji. Frontend musi obsługiwać stany błędów (np. odmowa dostępu, nieudane logowanie) i przekazać komunikaty użytkownikowi.

Dashboard twórcy (panel administracyjny): Dostępny po zalogowaniu. Zaimplementowany jako aplikacja typu SPA (Single Page App) dla płynności. Kluczowe widoki:

Strona główna panelu (podsumowanie): pokazuje saldo, ostatnie napiwki, ewentualne alerty (np. „Zweryfikuj email” lub „Uzupełnij profil”).

Historia transakcji: tabela z wszystkimi otrzymanymi napiwkami, filtrami po dacie/kwocie, możliwością eksportu CSV.

Wyплаты: formularz do zlecenia wypłaty – wybór kwoty, wybór metody (adres krypto lub konto bankowe). Po zleceniu pokazuje się status wypłaty (np. „przetwarzane” / „zrealizowane”).

Edycja profilu: upload avataru i banneru (integracja z np. usługą storage S3, generowanie miniaturek), zmiana opisu, linków do social mediów, ustawienie aliasu (alias może być generowany na podstawie logowania, np. z Google name, ale można go zmienić jeśli unikalny).

Ustawienia konta: zarządzanie preferencjami (np. powiadomienia e-mail on/off), wgląd w powiązane konta (lista usług z których użytkownik się logował – np. połączył konto Google i Twitch – możliwość odłączenia), zmiana ustawień bezpieczeństwa (dodanie 2FA, zmiana adresu email kontaktowego, itp.).

Narzędzia twórcy: wygeneruj kod widgetu (pojawia się snippet HTML/JS z jego unikalnym ID do wklejenia na stronę), podejrzuj jak wygląda Twój publiczny profil (podgląd), ustawienia celów i subskrypcji (jeśli dostępne).

Strona profilu twórcy (widok publiczny): To strona widoczna dla każdego fana pod adresem tipjar.com/@alias. Zawiera:

Banner i zdjęcie twórcy, nazwę i opis.

Jeśli ustawiony jest cel zbiórki – pasek postępu (np. 250/1000 USDC zebrane).

Główny element: formularz wsparcia – pole kwoty, opcja wiadomości, przyciski płatności (np. „Zapłać USDC”, „Karta Visa/MC”). Jeśli fan jest zalogowany w TipJar (rzadszy przypadek), może mieć dodatkowe opcje (np. użyj mojego salda TipJar).

Sekcja społecznościowa: lista ostatnich napiwków (np. „Anna: 5 USDC – Super film!” jeśli użytkownik nie był anonimowy i dodał komentarz). Można tu zastosować paginację lub ograniczyć do np. 10 ostatnich wpisów.

Linki do social mediów twórcy, jeśli podał (ikony YouTube, Twitter itd. – by fan mógł łatwo odwiedzić).

Komponent widgetu napiwków: Jest to uproszczona wersja formularza wsparcia, przeznaczona do osadzenia np. na stronie twórcy poza TipJar. Może mieć formę <iframe>

lub skryptu dodającego odpowiedni element DOM. Widget pokazuje np. przycisk „Wesprzyj [nazwaTwórcy] przez TipJar”, który po kliknięciu wyświetla okienko modalu z tym samym formularzem co na profilu. Realizuje to API TipJar poprzez JavaScript postMessage lub inny mechanizm komunikacji z rodzicem.

Interakcja z MetaMask/portfelami Web3: Frontend (profil lub widget) wykrywa, czy użytkownik ma zainstalowany portfel (np. window.ethereum). Jeśli fan wybiera „Zapłać USDC (krypto)”, a portfel jest dostępny, aplikacja:

1. Prosi użytkownika o połączenie się (Metamask poprosi o zgodę na udostępnienie adresu).
2. Pobiera adres aktywnego konta oraz sieć. Jeśli sieć nie jest obsługiwana (np. fan jest na Ethereum, a twórca preferuje Polygon), może poprosić o przełączenie sieci lub skorzystać z CCTP przez backend.
3. Wywołuje odpowiednie funkcje – np. przygotowuje transakcję ERC20 transfer do adresu portfela twórcy. Może tu być wykorzystana biblioteka ethers.js lub viem. Aplikacja wczytuje ABI tokenu USDC (lub korzysta z gotowej instancji kontraktu) i wywołuje `contract.transfer(tworcaAdres, kwota)`, inicjując popup Metamask do akceptacji.
4. Po potwierdzeniu wysyła transakcję do sieci – front może nasłuchiwać na jej potwierdzenie (promis z ethers, albo poll status). Równolegle może powiadomić backend (żeby backend też oczekiwał na tę transakcję, ewentualnie zapisując tymczasowo „tip pending”).
5. Gdy transakcja ma potwierdzenie (min. 1 block), frontend wyświetla sukces. Backend również wykrywa zdarzenie i finalizuje zapis napiwku.

Obsługa stanów i błędów: Aplikacja frontendu utrzymuje stan użytkownika (czy zalogowany, czy nie – np. przez context/Redux z informacją o tokenie JWT). W razie utraty sesji czy błędu API – przekierowuje do logowania. Błędy płatności (np. odrzucona karta, brak środków w portfelu fana) są czytelnie komunikowane w UI.

Od strony technicznej front-end wykorzystuje:

Next.js – do SSR strony publicznej (SEO) i jako wygodny framework do budowy SPA części zalogowanej.

TypeScript – zapewnia typowanie w całej aplikacji.

Biblioteki UI/UX: np. Tailwind CSS do szybkiego stylowania zgodnie z design system (paleta kolorów TipJar), ewentualnie komponenty z bibliotek (Chakra UI lub Material-UI jako baza, customizowana do stylu TipJar).

Zarządzanie stanem: W panelu twórcy można użyć lekkiej biblioteki typu Zustand lub kontekstu React. Dla prostszych potrzeb Redux Toolkit może być użyty, choć może to być nadmiarowe – zależy od złożoności stanu.

Komunikacja z backendem: Standardowe wywołania REST API przy użyciu fetch lub biblioteki Axios. Wrażliwe operacje (np. pobieranie listy transakcji) zawierają token JWT w nagłówku Authorization: Bearer Frontend również obsługuje cookies (jeśli JWT przekazywany jest w HttpOnly cookie po OAuth, Next.js API routes mogą pomóc w jego odczytaniu).

Biblioteki web3: Do interakcji z blockchainem używamy ethers.js lub nowszej viem – do obsługi podpisów (SIWE), wywołań kontraktów (transfer USDC) i integracji z Ethereum providerem.

Obsługa animacji i interakcji: Dla lepszego UX, stosowane są drobne animacje – np. animowane przyciski, płynne przewijanie. Można użyć biblioteki Framer Motion lub prostszych CSS transitions. W szczególności ważne jest informowanie użytkownika o trwających operacjach: np. gdy płatność jest w toku, przycisk zmienia się w spinner z komunikatem „Przetwarzanie...”.

Procesy Płatności i Integracja z Circle API

Jednym z kluczowych elementów TipJar są procesy związane z obsługą płatności – od momentu inicjacji przez fana, aż do odbioru środków przez twórcę. Poniżej przedstawiamy typowe scenariusze i to, jak zostały zaimplementowane z wykorzystaniem API Circle i technologii blockchain:

1. Inicjacja napiwku przez fana

Płatność kryptowalutą (USDC on-chain): Fan wybiera opcję zapłaty krypto. Frontend wykrywa dostępność portfela Web3:

Jeśli fan nie ma portfela lub odmawia połączenia – wyświetlana jest informacja jak założyć portfel (np. o MetaMask) lub wybór innej metody (karta).

Jeśli portfel jest połączony:

Frontend sprawdza, czy jest na właściwej sieci (np. Polygon). Jeśli nie, proponuje automatyczne przełączenie sieci w MetaMask (wysyłając `wallet_switchEthereumChain` z `chainId` Polygon).

Frontend wysyła zapytanie do backendu `/tips/initiate` z informacją: który twórca, jaka kwota, metoda „on-chain”. Backend w odpowiedzi generuje (lub pobiera) adres portfela twórcy w

wybranej sieci (z bazy, pole `mainWalletAddress` dla usera twórcy). Może również wygenerować unikalny `referenceId` transakcji (by potem sparować wpływ).

Frontend następnie wywołuje metodę kontraktu USDC: `transfer(to = adresTwórcy, amount = kwota)` poprzez `ethers.js`. Użytkownik zatwierdza transakcję, płaci ewentualną opłatę gazową ze swojego portfela.

Gdy transakcja zostanie nadana, frontend może przekazać hash transakcji do backendu (opcjonalnie). Backend albo już sam nasłuchuje blockchain (np. subskrypcja via `Alchemy/Infura`), albo czeka na webhook z Circle (jeśli portfel twórcy jest u Circle, to Circle może oferować webhook o depozycie), albo odpytuje co pewien czas listę transakcji portfela.

Po potwierdzeniu transakcji on-chain (zwykle w ciągu kilkunastu sekund na Polygon), backend odnotowuje w bazie, że tip został zrealizowany (ustawia status `Confirmed`, zapisuje hash).

Twórca dostaje powiadomienie (realtime lub e-mail). Fan na froncie już wcześniej zobaczył potwierdzenie (po 1 conf).

Gas Station: W tym scenariuszu fan sam zapłacił za gas (transakcja z jego portfela). Alternatywnie, gdyby fan korzystał z portfela Circle, mechanizm Gas Station spowoduje, że TipJar pokryje koszt: np. gdy backend wywołuje `circle.transfer()` z portfela fana do twórcy, a portfele są typu SCA, Circle automatycznie rozliczy gas w tle (lub obciąży konto platformy).

Płatność z portfela TipJar (transfer w Circle): Ten wariant zakłada, że fan jest zalogowany i posiada własny portfel utworzony w TipJar (Circle). Wtedy:

Fan wybiera np. kwotę i klika „Wyślij napiwek” – bez przechodzenia przez MetaMask, bo fan ma środki w swoim portfelu (np. doładował wcześniej).

Frontend woła autoryzowany endpoint `/tips/transfer` z kwotą i docelowym twórcą. Backend weryfikuje JWT fana, pobiera jego `circleWalletId` oraz `circleWalletId` twórcy.

Backend wywołuje metodę `Transfer` z API Circle: wskazuje portfel źródłowy fana, portfel docelowy twórcy i kwotę. Dodatkowo przekazuje `idempotencyKey` i np. `transferRequestId` do powiązania w systemie.

Circle od razu (lub po chwili) zwraca potwierdzenie dokonania transferu USDC wewnątrz ich systemu – środki stają się dostępne w portfelu twórcy praktycznie natychmiast. Ponieważ to transfer w obrębie jednej platformy custodial, nie ma fizycznej transakcji on-chain (lub jest to ujęte w wewn. księgach Circle, ewentualnie netting).

Backend oznacza transakcję jako wykonaną i informuje obie strony. Gas Station tu nie występuje, bo nie było on-chain tx od strony użytkownika (ew. Circle może w tle zrobić jedną zbiorczą transakcję po zsumowaniu wielu transferów).

Płatność kartą (fiat on-ramp): Fan wybiera opcję płatności kartą:

Frontend może przekierować fana do hostowanego checkoutu Circle (gdzie wpisze dane karty), albo wyświetlić własny formularz i wywołać API Circle Payments (przekazując token karty uzyskany np. z elementu Drop-In).

W obu przypadkach, Circle zajmuje się autoryzacją płatności. Po jej pomyślności:

Circle automatycznie zasila określony portfel USDC wskazany w zleceniu – tu wskazujemy portfel twórcy. Można więc wywołać `PaymentIntent` z parametrem `destinationWalletId` (jeśli Circle pozwala) lub zrobić to w dwóch krokach: najpierw utworzyć płatność na portfel własny platformy, a potem transfer na twórcę.

Backend TipJar otrzymuje webhook o statusie płatności (np. `payment.confirmed`) z identyfikatorem, kwotą itp. Na tej podstawie znajduje odpowiadającą transakcję i uznaje napiwek. Twórca dostaje powiadomienie, fan ewentualnie e-mail z podziękowaniem.

Ta ścieżka wymaga spełnienia pewnych wymogów regulacyjnych (TipJar pełni tu rolę pośrednika finansowego). Ale dla mikrotransakcji na początek można to uprościć – np. limitować kwoty fiat lub korzystać z licencji partnera.

2. Zarządzanie portfelami i sponsorowanie gazu (Circle Wallets & Gas Station)

Jak wspomniano, przy rejestracji twórcy TipJar backend wywołuje `createWallet` w Circle. Circle tworzy custodial wallet przypisany platformie (TipJar jest właścicielem technicznym portfela, twórca jest beneficjentem). Dzięki temu TipJar ma możliwość wykonywać operacje w imieniu twórcy poprzez API – np. inicjować wypłaty czy podpisywać transakcje. Portfele są typu SCA (Smart Contract Account) co oznacza, że transakcje z tych portfeli mogą być opłacane przez zewnętrznego sponsora (czyli platformę).

Gas Station/Paymaster: Dla portfeli SCA na sieciach EVM (np. Polygon, Ethereum) Circle oferuje mechanizm Gas Station – TipJar deponuje pewną ilość MATIC (dla Polygon) na koncie gas, z której to puli pokrywane są opłaty transakcyjne portfeli użytkowników. W praktyce:

Gdy TipJar zleca np. wypłatę 50 USDC z portfela twórcy na zewnętrzny adres Ethereum, Circle automatycznie użyje gas station – czyli rozliczy opłatę np. 0.005 MATIC, odejmując ją z puli platformy. Twórca nie musiał posiadać MATIC ani zajmować się tym.

Podobnie, jeśli fan nie ma ETH/MATIC na opłacenie transferu do twórcy, mechanizm Pay User's Gas in USDC spowoduje, że TipJar może z tego tytułu pobrać równowartość opłaty w USDC (np. 0.02 USDC) z portfela fana zamiast wymagać tokenów gas. W modelu napiwków często platforma może znieść ten koszt (bo jest niewielki) w ramach swoich 1-2% prowizji.

Cross-Chain Transfer (CCTP): W przypadku, gdy fan i twórca działają na różnych sieciach (np. fan ma USDC na Ethereum, ale twórca standardowo na Polygon), TipJar może użyć Circle CCTP do przeniesienia USDC między sieciami. CCTP to protokół spalania USDC na jednej sieci i mintowania na drugiej, sterowany przez Circle. W praktyce:

Fan inicjuje napiwek np. 10 USDC, wskazując że płaci ze swojego portfela Ethereum. Twórca preferuje Polygon.

TipJar przyjmuje płatność fana na Ethereum (on-chain, fan płaci gas), po czym backend automatycznie zleca CCTP transfer – 10 USDC jest spalane na Ethereum, Circle notyfikując to, a następnie 10 USDC jest emitowane na Polygon do portfela twórcy (który jest custodial w Circle i ma adres na obu sieciach, lub platforma określa docelowy chain).

Całość dzieje się dość płynnie, choć trwa dłużej niż zwykły transfer na jednej sieci (potrzeba finalizacji transakcji spalania). Jednak fan i tak widzi tylko że zapłacił, a twórca po chwili dostaje środki już na preferowanym chainie.

Monitoring i obsługa wyjątków: Wszystkie operacje płatności są monitorowane – jeśli jakkolwiek krok się nie powiedzie (np. transakcja on-chain nie doszła, API Circle zwróciło błąd) – system loguje to i podejmuje próby naprawcze:

W razie niepowodzenia on-chain TipJar może np. powiadomić fana: „Transakcja nie została potwierdzona, spróbuj ponownie” (lub jeśli środki wyszły ale nie dotarły – tu rzadkie, raczej finalność jest pewna po potwierdzeniu bloków).

Jeśli API Circle odmówi (np. limit dzienny, albo portfel nie utworzony jeszcze) – system może odłożyć operację do kolejki i spróbować później, informując użytkownika że transakcja jest opóźniona.

Bardzo ważne jest logowanie audytowe – w razie sporu można prześledzić całą ścieżkę: od żądania fana, przez zapisy w DB, po transakcje Circle i blockchain. Dzięki integracji z Circle dysponujemy unikalnymi ID operacji (transferId, paymentId itp.) które są przechowywane w naszej bazie dla późniejszego wglądu.

3. Wypłata środków przez twórcę

Twórca w panelu zleca wypłatę X USDC. Może podać adres swojego portfela zewnętrznego (np. własny MetaMask na Ethereum) lub konto bankowe.

Jeśli to wypłata na portfel krypto:

Backend otrzymuje żądanie /payout z kwotą i adresem. Sprawdza, czy twórca ma wystarczające saldo (w bazie lub pytając Circle o saldo portfela).

Następnie wywołuje `CircleService.transferUSDC(circleWalletId, externalAddress, amount)`. W przypadku Circle API, transfer na zewnętrzny adres może wymagać innego endpointu (np. Payouts API albo Transfers z flagą external). Circle wtedy:

Inicjuje transakcję on-chain z portfela twórcy na wskazany adres. Dzięki Gas Station, TipJar pokrywa opłatę (lub odlicza z kwoty, ale raczej platforma pokrywa).

Zwraca status (początkowo „pending”). Circle może dostarczyć webhook, gdy transakcja ma hash i gdy zostanie potwierdzona.

TipJar backend oznacza wypłatę w bazie i może od razu ująć środki (zamrozić je). Po potwierdzeniu finalnym oznacza wypłatę jako zrealizowaną.

Twórca dostaje potwierdzenie e-mail i/lub notyfikację „Wyplacono X USDC na adres ...”.

Jeśli to wypłata fiat (na konto bankowe):

Wymaga integracji z usługą payout od Circle lub innego operatora. Zwykle twórca musiałby wcześniej dodać swoje konto bankowe jako „beneficiary” (co może wymagać weryfikacji jego tożsamości – to już głębsza integracja KYC/AML).

Po zleceniu, backend wywołuje np. Circle Payouts API przekazując kwotę, walutę docelową, identyfikator beneficjenta. Circle wykonuje konwersję USDC->USD i zleca przelew na wskazane konto.

Proces ten trwa dłużej (np. 1-2 dni bankowe). TipJar w panelu twórcy pokazuje wypłatę jako „w toku”.

Gdy dostanie potwierdzenie (webhook o sukcesie), oznacza wypłatę jako zrealizowaną.

Limity i bezpieczeństwo wypłat: Można wprowadzić minimalną kwotę wypłaty (np. 10 USDC) by unikać zbyt drobnych transferów. Także zabezpieczenia: jeśli wykryto podejrzaną działalność (np. bardzo wiele napiwków z anonimowych źródeł w krótkim czasie), wypłata może wymagać dodatkowej autoryzacji manualnej przez admina.

Opłaty za wypłatę: TipJar może przerzucić koszt wypłaty na twórcę (np. jeśli wypłaca on-chain, to albo zmniejszyć kwotę o opłatę sieci – choć Gas Station to pokrywa, więc raczej nie – albo doliczyć symboliczną prowizję). To kwestia modelu biznesowego – do decyzji.

Design System (UI/UX)

Warstwa interfejsu TipJar została zaprojektowana tak, aby odzwierciedlać nowoczesny charakter technologii blockchain, a jednocześnie być przyjazna i intuicyjna dla użytkowników

spoza świata krypto. Projekt graficzny kładzie nacisk na czytelność, prostotę nawigacji i atrakcyjne elementy wizualne podkreślające ideę napiwków. Poniżej opis najważniejszych założeń design systemu:

Motyw kolorystyczny: Po przeanalizowaniu różnych palet, zdecydowano się na połączenie ciemnego turkusu (#008080) z kolorem złotym (#FFD700) jako barwy przewodnie TipJar.

Turkus symbolizuje zaufanie, stabilność i nowoczesność (kojarzy się z technologią blockchain, ale jest przyjaźniejszy niż chłodny niebieski). Tło aplikacji często jest bardzo ciemnym turkusem lub granatem (#1A1A2E) – co daje eleganckie, lekko futurystyczne płótno.

Złoto reprezentuje wartość i nagrodę – idealnie pasuje do idei napiwku jako docenienia twórcy. Używane jest na elementach akcji (CTA), ikonach monet, obramowaniach elementów interaktywnych. Działa jako kolor akcentowy przyciągający wzrok do najważniejszych miejsc (np. przycisk „Wesprzyj” jest złoty, co komunikuje „to jest cenne działanie”).

Barwy uzupełniające: biel (#FFFFFF) dla tekstu na ciemnych tłach (wysoki kontrast dla czytelności), odcienie szarości (#E0E0E0) do mniej istotnych elementów/ikon, jasny turkus lub mięta (#00C4FF) jako ewentualny drugi akcent (np. dla linków).

Typografia: Użyto nowoczesnych sans-serifów – np. Montserrat lub Inter – które dobrze wyglądają zarówno w nagłówkach, jak i dłuższym tekście. Nagłówki są pogrubione (Montserrat Bold dla przyciągnięcia uwagi). Tekst paragrafów jest prosty, czytelny (Inter Regular). Czcionki są skalowane responsywnie, by na urządzeniach mobilnych pozostały czytelne.

Styl ogólny: Nowoczesny, lekko futurystyczny. Interfejs jest dość minimalistyczny – dużo pustej przestrzeni (whitespace), wyraźne podziały sekcji. Elementy mają zaokrąglone krawędzie (przyciski, karty profili) nadając przyjaznego charakteru.

Delikatne ozdoby nawiązujące do blockchain: np. tło niektórych sekcji ma dyskretny wzór sieci bloków lub geometrycznych linii. Ikony mogą być stylizowane – np. symbol USDC, ikona portfela – z dodanymi neonowymi obrysami lub 3D efektem.

Grafiki i multimedia: Strona główna oraz profil twórcy mogą wykorzystywać grafiki 3D – np. animowana moneta USDC obracająca się, strzałka symbolizująca przepływ napiwku. Te animacje przyciągają uwagę i tłumaczą ideę (moneta → twórca). Ważne, by były responsywne – na dużym ekranie pełen detal, na małym uproszczone.

Możliwe jest użycie bibliotek typu Lottie lub Three.js do osadzenia takich animacji wektorowych/3D bez obciążania strony.

Animacje i interakcje: Subtelne mikro-animacje poprawiają UX:

Przyciski CTA: przy najechaniu delikatnie zmieniają odcień lub pokazują efekt połysku (jak przesuwające się światło po złotym przycisku).

Ładowanie danych: zamiast pustego ekranu, użyto animacji ładowania – np. kręcące się kółko w kolorze złotym (stylizowane jak moneta) sygnalizuje, że dane się ładują.

Pojawianie się elementów przy scrollu: sekcje strony głównej (np. „Jak to działa?” z ikonkami) pojawiają się z lekkim opóźnieniem, z animacją fade-in lub slide-up, by dodawać dynamiki podczas przewijania.

Powiadomienia w panelu: np. gdy nowy napiwek wpłynie w czasie rzeczywistym, może pokazać się wysuwany toast z informacją („+1 USDC od Jan Kowalski!”) – z animacją wjazdu i znikania.

Układ i responsywność: Każdy ekran projektowano w myśl mobile-first – na telefonach układy są jednokolumnowe, z dużymi dotykowymi przyciskami. Na desktopie często stosowane są dwie kolumny:

Np. sekcja hero na landing page: po lewej tekst i CTA, po prawej duża grafika 3D.

Sekcja „Jak to działa”: trzy kolumny z ikonami i opisami kroków (na mobile zamienia się to w pionowy stack).

Karty twórców (carousel) widoczne obok siebie na szerokim ekranie, a przewijane pojedynczo na małym.

Panel twórcy: menu boczne (sidebar) na dużym ekranie, a na mobilnym hamburger menu i ukrywana nawigacja.

Komponenty UI:

Przyciski: Dominujące są warianty: złote tło + turkusowy tekst (główne akcje), oraz odwrotnie – turkusowe tło + złoty tekst (drugorzędne akcje). Wszystkie mają zaokrąglone rogi i lekki cień. Rozmiar raczej duży (łatwe do kliknięcia). Ikony na przyciskach (np. logo Google obok „Zaloguj przez Google”) są w odcieniu złota lub białe na ciemnym tle.

Pola i formularze: Pola tekstowe na ciemnym tle są ciemnoszare z jasnym tekstem i złotymi obramówkami focus. Etykiety pól – jasny turkus lub szarość. Formularz napiwku jest maksymalnie uproszczony: suwak lub plus/minus do wyboru kwoty, pole komentarza, checkbox „anonimowo”. Wszystko duże i czytelne.

Karty i listy: Np. lista historii napiwków – na desktopie tabela, ale na mobile jako karty jedna pod drugą. Kartę/wiersz odziela cienka złota linia. Każdy wpis zawiera tekst w turkusie (np. „Anna – 5 USDC – Thanks!”) na jasnym tle. Karty twórców (na stronie głównej) mają ciemnoturkusowe tło, zaokrąglone złote obramowanie i dynamicznie pokazującą się wartość („\${suma} USDC zebrane”).

Modalne okienka: np. po kliknięciu „Wesprzyj” na widgetcie – pojawia się modal z ciemnym tłem i złotymi obramowaniami. Przy zamknięciu modala jest animacja zanikania.

Ikonografia: Oprócz logo USDC (które jest niebieskie – tu można je tonować na turkusowo, aby pasowało), używane są ikony reprezentujące:

Portfel, monetę, strzałki przepływu – np. ikona portfela może mieć złoty kolor, ikona monety turkusowy akcent.

Ikony mediów społecznościowych w stopce – jednolite białe, z efektem hover zmieniającym na turkus.

Własne logo TipJar – powinno być proste i rozpoznawalne: np. stylizowana ikona słoika (jar) ze znakiem \$ lub monetą. W wersji kolorystycznej: złota moneta na tle turkusu lub odwrotnie. Logo pojawia się w navbarze oraz na materiałach marketingowych.

Dostępność (UX): Zwrócono uwagę na:

Kontrast kolorów tekstu do tła (WCAG) – np. złoty na turkusie jest dobrze widoczny, biały na turkusie też. Unikamy mało kontrastowych kombinacji.

Teksty alternatywne dla ikon/grafik – np. jeśli jest animowana moneta, dodajemy alt="animacja monety USDC" dla dostępności.

Możliwość obsługi klawiaturą – wszystkie elementy interaktywne (przyciski, linki) są fokusowalne i sterowalne bez myszy.

Tłumaczenia/internacjonalizacja – od początku teksty w aplikacji są trzymane w plikach językowych, co ułatwi wprowadzenie wielu języków (istotne, bo platforma globalna).

Podsumowując, UI TipJar stara się łączyć klimat krypto (technologia, nowoczesność) z uczuciem nagrody i wartości (złote akcenty), jednocześnie nie odpychając mniej technicznych użytkowników – dlatego interfejs jest przejrzysty, przypomina raczej nowoczesną aplikację finansową/fintech niż stronę dla programistów blockchain.

Projekt Mobilny

Aplikacja mobilna TipJar jest przewidziana głównie z myślą o twórcach, aby mogli zarządzać swoimi profilami i środkami z dowolnego miejsca, otrzymywać powiadomienia push o nowych napiwkach, itp. Ewentualnie może ona też obsługiwać fanów (przeglądanie profili i wspieranie), choć na starcie zakładamy, że fani skorzystają raczej ze stron www lub widgetów, a dedykowana aplikacja mobilna może pojawić się później jako rozszerzenie społecznościowe (np. przeglądaj listę twórców w aplikacji, odkrywaj nowych i wspieraj ich – to potencjalny kierunek).

Technologia: Rozważane są cross-platform rozwiązania:

React Native – naturalny wybór, biorąc pod uwagę, że web frontend jest w React. Pozwoli współdzielić część logiki (np. definicje modeli, serwisy API). React Native + Expo może przyspieszyć start.

Flutter – alternatywa zapewniająca świetny wygląd i wydajność, ale wymagająca pisania od zera UI w Dart. Zespół web może potrzebować dodatkowych kompetencji.

Native (Swift/Kotlin) – raczej nie, ze względu na szybko zmieniające się wymagania i ograniczone zasoby, cross-platform wydaje się bardziej efektywny.

Funkcjonalności aplikacji mobilnej (dla twórcy):

Dashboard mobilny: Podobny zestaw informacji jak w wersji web, ale dostosowany do mniejszego ekranu. Priorytetem jest szybki wgląd: saldo, ostatnie napiwki (np. jako lista kart przewijanych pionowo), przycisk „Udostępnij profil” (by łatwo skopiować link lub pokazać QR komuś bezpośrednio z telefonu).

Powiadomienia push: Aplikacja, po zalogowaniu, rejestruje urządzenie do otrzymywania powiadomień (np. wykorzystując Firebase Cloud Messaging lub usługę Expo push). Dzięki temu twórca dostaje natychmiastowe powiadomienie push „ $\{fanName\}$ wsparł Cię kwotą 5 USDC!” – co jest bardzo cenną funkcją mobilną. Również powiadomienia o osiągnięciu celu zbiórki, o nowych funkcjach itp. mogą być wysyłane.

Skaner kodów QR: Aplikacja może zawierać skaner kodów QR – np. gdy twórca spotka fana na konwencji, może pokazać swój kod do zeskanowania lub fan może zeskanować kod twórcy, a aplikacja bezpośrednio rozpozna profil i umożliwi wsparcie. Ewentualnie fanowska część aplikacji mogłaby użyć skanera, ale w kontekście twórcy raczej nie – to byłaby bardziej funkcja aplikacji dla fanów.

Offline mode i synchronizacja: Przy słabszym Internecie (mobilnym) aplikacja powinna nadal wyświetlać ostatnio znane dane (cache ostatnich napiwków, salda) i odświeżyć po uzyskaniu połączenia.

Tryb twórcy vs. fana: Można przewidzieć, że aplikacja w przyszłości będzie dwufunkcyjna – podczas logowania wybierasz, czy jesteś twórcą czy fanem, i UI się dostosowuje. Fan mógłby przez aplikację śledzić ulubionych twórców i szybko im wysłać napiwki mobilnie (np. zintegrować Apple Pay / Google Pay do doładowania portfela TipJar i wspierania jednym kliknięciem). Jednak to dalekosiężna opcja.

UI/UX mobilny: Będzie spójny z web, ale dostosowany:

Ciemny motyw turkus-złoto również obowiązuje.

Nawigacja najpewniej dolnym paskiem (tab bar) – np. zakładki: Napiwki (główne info), Profil (edytuj profil), Powiadomienia, Ustawienia.

Ekran będą scrollowalne bez przeładowań, płynne interakcje dotykowe (React Native zapewnia animacje, można użyć bibliotek typu Reanimated).

Przykładowy scenariusz mobilny: Twórca otrzymuje powiadomienie push, otwiera aplikację – na głównym ekranie widzi wyróżnioną informację o nowym napiwku (np. zielone tło dla nieprzeczytanych). Może kliknąć by zobaczyć szczegóły i ewentualnie odpowiedzieć fanowi (jeśli taka funkcja by istniała – np. wysłać szybką reakcję „Dziękuję!”). Na ekranie profilu może jednym tapnięciem skopiować link do profilu lub wyświetlić QR pełnoekranowo, by ktoś mógł zeskanować. W ustawieniach może włączyć/wyłączyć powiadomienia, ustawić konto bankowe do wypłat itp.

Publikacja i dystrybucja: Aplikacja mobilna będzie dostępna w App Store i Google Play. Z tego powodu kwestia zgodności z wytycznymi sklepów (szczególnie Apple) jest ważna – np. Apple ma restrykcje dot. aplikacji finansowych/kryptowalutowych i opłat in-app. Tutaj potencjalnie napiwki mogą być uznane za płatności peer-to-peer i raczej nie podpadają pod model IAP (wewnątrzaplikacyjny zakup Apple), ponieważ środki nie służą do cyfrowej treści w aplikacji, a idą poza nią. Niemniej, należy przygotować się do argumentacji w razie weryfikacji (podkreślić zdecentralizowany charakter transakcji).

Podsumowując, aplikacja mobilna zwiększy zaangażowanie twórców, dając im narzędzie do bycia na bieżąco ze wsparciem fanów, a w przyszłości może stać się też platformą skupiającą społeczność fanów wokół TipJar.

Środowisko Developerskie, CI/CD i Infrastruktura

Aby zapewnić sprawny rozwój oraz stabilne wdrażanie platformy TipJar, dużą wagę przyłożono do przygotowania odpowiedniego środowiska developerskiego i praktyk DevOps. Poniżej opis konfiguracji i narzędzi:

Środowisko programistyczne (Dev Environment)

Monorepo vs repozytoria rozdzielone: Projekt może być utrzymywany jako monorepo (np. przy użyciu Nx lub TurboRepo) zawierające frontend, backend, kontrakty itp. lub osobne repo dla frontendu i backendu. Monorepo ułatwia współdzielenie typów (np. definicje modelu Tip, User w TypeScript mogą być wspólne) i zapewnia spójne wersjonowanie.

Docker Compose: Przygotowany jest plik docker-compose.yml ułatwiający uruchomienie wszystkich niezbędnych usług lokalnie – np. bazy danych Postgres, ewentualnie kontenera z narzędziem Redis (dla kolejek), a także samej aplikacji backendowej. Frontend w dev może chodzić na żywo przez npm run dev, ale docelowo też może mieć swój Dockerfile.

Hot-reload i debug: NestJS w trybie dev nasłuchuje zmian (ts-node z webpackem), React Next.js również posiada hot reload. Programiści mogą debugować backend poprzez VSCode attach (Nest startuje z flagą —inspect). Dzięki Docker Compose można też debugować wewnątrz kontenera.

Dostęp do API zewnętrznych: Plik konfiguracyjny .env (nie commitowany) przechowuje klucze: np. CIRCLE_API_KEY, CIRCLE_ENTITY_ID, GOOGLE_OAUTH_CLIENT_ID, GOOGLE_OAUTH_SECRET, JWT_SECRET itp. Dla dev używane są klucze do sandboxów (Circle oferuje sandbox environment, podobnie można utworzyć testowe projekty OAuth Google/Twitch).

Baza danych dev: Postgres lokalnie w Dockerze. Migracje schematu zarządzane przez Prisma (polecenia npx prisma migrate dev itd.). Można generować przykładowe dane (seedy) – np. kilku testowych twórców i fanów.

Kontrola wersji i CI/CD

Repozytorium Git: Cały kod utrzymywany jest w systemie kontroli wersji (GitHub lub GitLab). Obowiązują code review dla wszystkich zmian (pull requesty / merge requesty muszą zostać zaakceptowane przed połączeniem do gałęzi głównej).

Ciągła integracja (CI): Skonfigurowano pipeline (np. GitHub Actions lub GitLab CI):

Przy każdym pushu/pr (szczególnie na branch główny main lub dev), uruchamiane są testy jednostkowe i integracyjne.

CI także wykonuje linting (ESLint dla TS, stylelint dla CSS) i budowanie aplikacji (czy kompilacja przechodzi).

Użyto też narzędzi do skanowania zależności (np. Dependabot automatycznie tworzy PR z aktualizacjami paczek JS, a npm audit lub Snyk sprawdza podatności).

W CI można generować również preview deployments – np. dla frontendu generować wersję podglądową (Vercel preview dla każdej gałęzi PR).

Ciągłe dostarczanie (CD): Proces wdrażania na środowiska:

Staging: Każda zmiana po zmergowaniu do main automatycznie trafia na środowisko testowe (staging). Tu wykorzystywane są np. AWS Elastic Beanstalk / Docker do wgrania nowej wersji backendu, a frontendu na np. Vercel (dla spójności można też hostować front w S3/CloudFront czy podobnie). Baza danych staging jest odrębna (często kopia produkcyjnej z anonimizowanymi danymi lub zupełnie testowa). Staging jest miejscem do testów manualnych QA oraz dla wybranych beta-testerów.

Production: Wdrożenie na produkcję jest również zautomatyzowane, ale wymaga zatwierdzenia (manual approval) – np. release manager zatwierdza pipeline, który aktualizuje infrastrukturę produkcyjną. Tutaj kluczowe jest zachowanie migracji DB (CI/CD najpierw wykonuje migracje, potem wprowadza nowe serwisy).

Roll back: Pipeline posiada mechanizmy rollback (np. w AWS utrzymanie poprzedniej wersji kontenerów, by móc szybko przełączyć ruch w razie krytycznego błędu).

Containerization i orkiestracja:

Obrazy Docker są budowane w CI dla backendu (np. Dockerfile oparte na node:18-alpine, z komendą do uruchomienia `npm run start:prod`). Dla frontendu Next.js budowany jest static export lub image z serwerem node (w zależności od deploy).

W produkcji można użyć Kubernetes (np. GKE/EKS/AKS) albo prostszego podejścia: AWS ECS Fargate czy Heroku/Render dla mniejszej skali. Wybór zależy od przewidywanego obciążenia i zasobów zespołu DevOps. W początkowej fazie możliwe jest użycie np. platformy Heroku lub Render.com dla szybkości – one integrują CI i zapewniają prosty scaling. Jednak docelowo, dla pełnej kontroli i kosztów, przejście na własną infrastrukturę (np. AWS) jest planowane.

Baza danych produkcyjna – np. AWS RDS (PostgreSQL) z repliką read-only i automatycznym backupem co 24h.

Sekrety w produkcji przechowywane w AWS Secrets Manager lub odpowiedniku (Google Secret Manager, HashiCorp Vault), a do kontenerów trafiają przez zmienne środowiskowe podczas deploymentu.

Monitoring i Logging:

Logi aplikacyjne: W kontenerach logi są zbierane przez system (stdout). Skonfigurowano centralizację logów – np. ELK stack (Elasticsearch + Kibana) lub prostsze Papertrail/Datadog logs. Umożliwia to filtrowanie logów po poziomie (info, warn, error) i szybkie debugowanie problemów.

Monitoring metryk: Zaimplementowano podstawowe metryki w backendzie (np. liczba żądań, czasy odpowiedzi, użycie pamięci). Wdrożono Prometheus do scrapowania tych metryk oraz Grafanę do wizualizacji dashboardów. Mierzone są też specyficzne metryki biznesowe: liczba napiwków na minutę, wolumen USDC dziennie, itp.

Alerting: Ustawiono alerty (np. w Grafanie/Prometheus Alertmanager lub Datadog APM) – powiadomienia na Slack/email gdy: CPU serwera > 80%, pamięć > 90%, czas odpowiedzi API > określonego progu (np. 95 percentyl > 1s), liczba błędów 5xx wzrasta powyżej X na minutę, itp.

Sentry (monitoring błędów): Frontend i backend są zintegrowane z Sentry. Każdy nieobsłużony wyjątek w backendzie skutkuje wysłaniem eventu do Sentry z stack trace. Podobnie w frontendzie – błędy runtime (np. nieobsłużone Promise) trafiają do Sentry. To pozwala wychwycić problemy zanim zgłoszą je użytkownicy.

Testowanie:

Testy jednostkowe: Backend – Jest + supertest (np. testy kontrolerów API, serwisów z mockami zewnętrznych zależności). Frontend – Jest/React Testing Library dla komponentów UI.

Testy integracyjne: Można użyć podejścia z Testcontainers – uruchamiać w pipeline kontener Postgresa i np. testować rzeczywiste wywołania DB, czy integracje z zewnętrznymi API przy użyciu stubów. Testy integracyjne sprawdzają pełne scenariusze (rejestracja twórcy -> czy w bazie pojawił się user i czy wywołano utworzenie portfela w Circle).

Testy end-to-end: W miarę możliwości, wykorzystać framework Cypress lub Playwright dla testów E2E frontendu z backendem (np. symulacja użytkownika: otwiera stronę, loguje się przez stub Google, wysyła napiwek na sandboxie, sprawdza czy twórca widzi go). To wymaga sporo konfiguracji (np. stub autoryzacji OAuth – można stworzyć fałszywego providera testowego).

Testy wydajnościowe: Przy rosnącej skali, zaplanowane są testy obciążeniowe, np. narzędziem k6 lub Locust – symulacja tysiąca równoczesnych płatności w ciągu minuty, pomiar czy system nadąży i gdzie ewentualnie występują wąskie gardła.

Staging vs Production konfiguracja:

Staging używa Circle sandbox (oddzielny API key), testowego środowiska OAuth (np. Google OAuth aplikacja w trybie test), testowych smart kontraktów USDC (np. USDC na Mumbai testnetcie lub sandbox provided by Circle).

Production – wiadomo, prawdziwe klucze i endpointy. Należy dopilnować, by żadne dane sandboxowe nie przedostały się na produkcję.

Domena: staging może działać na subdomenie typu staging.tipjar.com.

W pipeline CI/CD tworzone są osobne container images tagowane (tipjar-backend:staging vs tipjar-backend:latest).

Bezpieczeństwo w CI/CD: Dostęp do środowisk (klucze deploy, hasła DB) są trzymane w secure storage CI. Każdy build i deploy jest audytowalny. Weryfikujemy też dependencies (Snyk) i mamy politykę odnawiania kluczy/API co jakiś czas. Regularnie (co kwartał) planujemy przeprowadzać testy penetracyjne i code review pod kątem bezpieczeństwa (można zlecić zewnętrznemu audytorowi, szczególnie przed launchem).

Dzięki takiemu podejściu DevOps, zespół jest w stanie szybko iterować (regularne deploje na staging, częste release na produkcję), utrzymując wysoką jakość i stabilność platformy.

Model Biznesowy

TipJar, jako platforma fintech dla twórców, planuje zarabiać w sposób przejrzysty i zrównoważony, tak by jednocześnie utrzymać przewagę konkurencyjną (niska prowizja) i pokryć koszty operacyjne oraz rozwoju. Główne założenia modelu biznesowego:

Prowizja od transakcji: Standardowym źródłem przychodu będzie potrącanie niewielkiej prowizji od każdego napiwku. Proponowany poziom to 1–2% od kwoty wsparcia. Czyli przy napiwku 5 USDC, platforma zatrzymuje 0,05–0,1 USDC, reszta trafia do twórcy. Taka stawka jest na tyle niska, że nie zniechęca twórców (otrzymują 98–99% wartości), a zarazem przy dużej liczbie transakcji może zapewnić pokrycie kosztów. (Warto wspomnieć, że koszty transakcyjne blockchain i opłaty za usługi zewnętrzne także muszą być z tego pokryte, więc dokładna stawka prowizji wymaga analizy finansowej. Możliwe, że dla mniejszych transakcji prowizja procentowa będzie wyższa minimalnie, by pokryć stałe koszty np. \$0.01 za transfer USDC itp.)

Model freemium / plany premium: Platforma może wprowadzić dodatkowe płatne opcje dla twórców:

Plan Premium dla Twórcy: Za stałą miesięczną opłatę (np. \$10/mies) twórca otrzymuje pewne bonusy: brak prowizji od transakcji (100% napiwków dla niego), większe możliwości personalizacji profilu (np. własny CSS, brak branding TipJar), wcześniejszy dostęp do nowych funkcji, wyróżnienie w katalogu twórców itp.

Funkcje dodatkowe za opłatą: Np. twórca może zakupić boost swojej strony w sekcji „Odkryj popularnych twórców”, lub zlecić projektowanie unikalnego widgetu (usługa premium).

Sklep z gadżetami cyfrowymi: to bardziej eksperymentalny pomysł – TipJar mógłby umożliwić twórcom sprzedawanie drobnych cyfrowych przedmiotów za USDC (np. ekskluzywne naklejki, klipy video z podziękowaniem, NFT kolekcjonerskie). Platforma pobierałaby prowizję od tych transakcji lub listing fee.

Odsetki od depozytów (yield): Jeśli platforma osiągnie znaczącą sumę przechowywanych środków (USDC na portfelach), może rozważyć generowanie przychodu od odsetek/yield na tych środkach:

USDC sam w sobie nie rodzi odsetek, ale są możliwości np. lokowania części środków w protokołach DeFi (bezpiecznych) lub programie odsetkowym Circle. Ostrożność: to niesie ryzyko i wymaga zgodności regulacyjnej (staje się to wtedy działalność quasi-bankowa). Na start można założyć, że TipJar nie inwestuje środków użytkowników, trzyma je 1:1 (co zwiększa zaufanie).

Partnerstwa i white-label: TipJar może oferować white-label swojej technologii dla innych platform:

Np. integracja TipJar jako modułu napiwków w serwisie streamingowym czy na platformie blogowej, za opłatą licencyjną lub podział przychodów.

Partnerstwa z sieciami MCN (Multi-Channel Network) lub agencjami twórców – TipJar ułatwia im monetyzację, a w zamian dzieli się prowizją lub pobiera opłatę za dostęp premium.

Koszty operacyjne: Obejmują:

Opłaty infrastrukturalne (serwery, API Circle – w obecnym modelu Circle pobiera opłaty za transakcje? W programowalnych portfelach płaci się głównie za gas, ewentualnie za użycie API jeśli przekracza darmowy tier).

Sponsorowanie gazu – to ważny element wydatków. Na szczęście na Polygon transakcja kosztuje ułamki centa, więc np. 1000 transakcji to powiedzmy \$1–2. Prowizja 1% z tych transakcji pewnie pokryje to z nawiązką. Przy dużej skali jednak koszty gas mogą rosnąć – stąd monitorowanie i ewentualne dostosowanie prowizji/progu minimalnego napiwku (np. 0.5 USDC minimum, by nie płacić gas dla super małych kwot).

Obsługa płatności kartą – procesor płatności (np. Stripe, Circle) pobiera swoją prowizję ~3%. To nie może być całkowicie przerzucone na TipJar przy 1% prowizji, bo to nierentowne. Dlatego możliwe rozwiązania: albo fan płacąc kartą widzi, że zapłaci np. \$5.20 by twórca dostał równowartość \$5 (czyli doliczamy 4% opłaty transakcyjnej), albo TipJar akceptuje mniejszą marżę przy płatnościach kartą. Do analizy – bo dużo fanów spoza krypto będzie chciało płacić kartą, więc nie należy zniechęcać ich dodatkowymi opłatami. Być może prowizja 1-2% dotyczy tylko płatności krypto, a dla fiat jest wyższa by pokryć koszt.

Wsparcie użytkowników, marketing, rozwój – stałe koszty firmy.

Skalowanie przychodów: Zakładając globalny zasięg i atrakcyjność oferty:

Nawet przy mikropłatnościach, duża liczba użytkowników może wygenerować znaczący wolumen. Np. 100 tys. napiwków miesięcznie o średniej wysokości \$3 daje \$300k przepływu – 1% z tego to \$3k dla platformy. Przy milionach użytkowników skala rośnie wykładniczo.

Dodatkowo, subskrypcje miesięczne (jeśli wdrożone) zapewniają powtarzalny dochód (a TipJar od każdej takiej subskrypcji też bierze prowizję).

Plany premium (jeśli np. 1000 twórców wykupi premium po \$10 = \$10k mies. przychodu stałego).

Konkurencja i przewagi: Istnieją inne systemy napiwków (np. wysyłanie krypto bezpośrednio, klasyczne platformy typu BuyMeACoffee). TipJar konkuruje niższą prowizją i integracją web3. Model biznesowy zakłada, że pewien % użytkowników i tak będzie preferował istniejące platformy, ale dynamiczny rozwój społeczności krypto i stablecoinów stwarza nową niszę. TipJar może także konkurować oferując np. program poleceń (referral) – fan zapraszający twórcę może dostać jednorazową nagrodę, itd. To jest element marketingu raczej niż modelu biznesowego, ale wspiera wzrost przychodu pośrednio.

Podsumowując, monetyzacja TipJar opiera się na małej marży od dużego wolumenu transakcji. Jest to model stosunkowo efektywny kosztowo (platforma opiera się na automatycznych mechanizmach, więc po początkowym nakładzie pracy, obsługa pojedynczych transakcji jest tania). Przy zdrowym wzroście użytkowników i utrzymaniu kosztów technologii w ryzach, nawet 1% prowizji może uczynić projekt zyskownym, jednocześnie dając twórcom więcej zarobku niż jakakolwiek alternatywa – co przyciągnie ich i ich fanów do korzystania z TipJar.

Elementy Gamifikacji

Aby zwiększyć zaangażowanie zarówno fanów, jak i twórców, TipJar planuje wprowadzić elementy grywalizacji do platformy. Gamifikacja ma na celu uczynienie procesu wsparcia bardziej satysfakcjonującym i zachęcającym do powtarzalności, a także budowanie społeczności wokół interakcji fan–twórca. Oto proponowane elementy gamifikacyjne:

Odnaki i osiągnięcia dla fanów: Fani mogą zdobywać odznaki za różne formy aktywności:

„Pierwszy napiwek” – za dokonanie pierwszej wpłaty w serwisie.

„Super Fan” – za przekroczenie pewnej łącznej kwoty wsparcia dla jednego twórcy (np. 100 USDC).

„Mecenas” – za wsparcie określonej liczby różnych twórców (np. 10).

„Stały bywalec” – za comiesięczne wspieranie kogoś przez 6 kolejnych miesięcy.

Odnaki będą widoczne opcjonalnie przy nazwie fana (jeśli nie jest anonimowy) – np. fan komentujący napiwek może mieć obok pseudonimu małą ikonę trofeum.

Rankingi i Top Fans: Na profilach twórców może pojawić się ranking Top 10 fanów (tych, którzy przekazali najwięcej USDC). To może być tygodniowy/miesięczny ranking albo ogólny. Taka tablica motywuje fanów do „rywalizacji” o bycie najhojniejszym. Oczywiście z uwagą, by nie zniechęcić drobnych darczyńców – być może twórca będzie mógł wyłączyć ranking, jeśli nie pasuje to do jego społeczności.

Cele i społeczna zbiórka: Cele finansowe twórcy (np. 1000 USDC na album muzyczny) same w sobie są gamifikacją – pokazują pasek postępu. Fani widząc, że cel jest blisko, mogą się zmobilizować. Dodatkowo planujemy mechanizm „challenge”: twórca może ogłosić, że jeśli cel zostanie osiągnięty przed czasem X, to zrobi coś specjalnego (np. bonusowy stream). To angażuje społeczność we wspólny wysiłek.

Interakcje społecznościowe przy napiwkach: Po wsparciu twórcy fan może mieć możliwość zostawienia tzw. „shout-out” – krótkiej wiadomości, która (za zgodą twórcy) będzie publicznie widoczna. Inni fani mogą „polajkować” te wiadomości lub reagować emotkami. Tworzy to element społecznościowy – coś jak mini-forum na profilu twórcy związane z donacjami.

Najciekawsze/śmieszne wiadomości (poparte większym napiwkiem) mogą zyskać uznanie innych.

Poziomy twórców: Gamifikacja może dotyczyć również twórców – np. wprowadzić poziomy zależne od zaangażowania na platformie:

Level 1: początkujący (0–50 USDC zebranych),

Level 5: zaawansowany (5000+ USDC zebranych).

Te poziomy mogłyby odblokowywać pewne funkcje lub po prostu być elementem prestiżowym (oznaczonym na profilu gwiazdkami czy innym symbolem). Dodatkowo twórcy z wyższymi poziomami mogliby otrzymywać bonusy od TipJar, np. darmowy miesiąc planu premium, wyróżnienie na stronie głównej, itp.

Wyzwania i kampanie czasowe: TipJar może organizować globalne eventy:

Np. „Tydzień nowych twórców” – fani wspierający w tym tygodniu debiutujących twórców dostają podwójne punkty do odznak albo twórcy dostają niższą prowizję w tym okresie.

„Challenge: 100k w 30 dni” – globalny licznik wszystkich napiwków, jeśli społeczność osiągnie dany próg, TipJar np. przekazuje część dochodu na cel charytatywny lub rozdaje limitowane odznaki uczestnikom.

Personalizacja i zabawa: Twórcy mogą personalizować swoje strony poprzez motywy kolorystyczne, tła – odblokowywane osiągnięciami lub w planie premium. Fani z kolei mogą wybierać wyświetlane avatary w swoich wiadomościach (np. NFT połączone z portfelem, co integruje środowisko krypto collectibles z TipJar – to przyszłościowe myślenie).

System punktów (lojalności): Można wprowadzić wirtualne punkty lub token (niekoniecznie on-chain) za aktywność. Np. za każdy 1 USDC napiwku fan dostaje 100 pkt „karma”. Punkty te mogłyby służyć do rywalizacji rankingowej albo do drobnych nagród – np. TipJar co miesiąc losuje nagrody rzeczowe lub cyfrowe wśród najbardziej aktywnych.

Grywalizacja procesu rejestracji: Nawet onboarding twórcy można potraktować z nutą gry – np. pasek postępu „Uzupełnij 100% profilu, aby zdobyć odznakę Verified i zyskać zaufanie fanów”. Daje to motywację do pełnego skonfigurowania konta.

Oczywiście, wprowadzając gamifikację, należy uważać, by nie przytłoczyć użytkowników i by nie stworzyć wrażenia, że „chodzi o wyścig pieniędzy”. Wszystkie mechanizmy muszą być przemyślane pod kątem społeczności – wielu twórców będzie dbać o to, by fani nie czuli się zmuszani do rywalizacji finansowej. Dlatego TipJar planuje dać twórcom kontrolę:

Możliwość wyłączenia lub ukrycia rankingu top fanów.

Moderacja treści w shout-outach (twórca może usunąć niestosowny komentarz fana).

Opcje prywatności – fan może również zastrzec, by nie brać udziału w rankingach (anonimowość lub incognito).

Docelowo jednak elementy grywalizacji powinny urozmaicić doświadczenie. Drobne osiągnięcia i odznaki mogą sprawić frajdę fanom, a twórcom dodatkowo podpowiedzieć, którzy fani są najbardziej zaangażowani. To wszystko buduje bardziej żywą społeczność wokół platformy, wykraczając poza czysty akt przekazania napiwku.

Prywatność i Zgodność z Regulacjami

Kwestie prywatności danych oraz zgodności prawno-regulacyjnej (compliance) są kluczowe przy projektowaniu TipJar, ze względu na obsługę płatności i danych użytkowników na skalę globalną. Poniżej przedstawiamy, jak platforma podchodzi do tych zagadnień:

Ochrona Danych Osobowych (GDPR/CCPA itp.)

Minimalizacja danych: TipJar zbiera tylko te dane, które są niezbędne do działania usługi. Przy logowaniu OAuth są to głównie: adres e-mail, nazwa profilu i ewentualnie URL awataru. Przy rejestracji Web3 – praktycznie brak danych osobowych (tylko adres portfela). Nie prosimy o adresy zamieszkania, numery dokumentów czy inne wrażliwe dane od zwykłych użytkowników.

Zgody i Polityka Prywatności: Podczas rejestracji twórca musi zaakceptować regulamin i politykę prywatności. Polityka jasno opisuje, jakie dane zbieramy i w jakim celu. Jeśli w przyszłości integrujemy jakiegokolwiek trackery/analitykę, zapewniamy opcję opt-out zgodnie z RODO (np. baner cookies).

Prawo do bycia zapomnianym: Użytkownicy (zwłaszcza twórcy) będą mieli możliwość usunięcia konta, co wiąże się z usunięciem ich danych osobowych z bazy (z wyjątkiem danych, które musimy zachować np. do rozliczeń finansowych – wtedy anonimizujemy je). Mechanizm usuwania musi również uwzględnić dane w zewnętrznych serwisach: np. jeśli user logował się Google, możemy usunąć jego tokeny; jeśli miał portfel Circle, być może nie usuniemy go od razu (ze względów księgowych), ale odłączymy.

Bezpieczeństwo przechowywania danych: Dane personalne w bazie (np. e-maile) są przechowywane w formie zaszyfrowanej lub przynajmniej zahaszowanej tam, gdzie możliwe. Dostęp do bazy jest ograniczony (zasada minimalnych uprawnień, IP whitelisting dla adminów). Backupy bazy są szyfrowane.

Przekazywanie danych poza UE: Jeśli TipJar będzie mieć użytkowników z UE (co prawie pewne), a serwery są np. w USA, trzeba zapewnić zgodność z GDPR odnośnie transferu danych. Najlepiej ulokować serwery i bazę przetwarzającą dane osobowe w obrębie EU (np. AWS Frankfurt) lub stosować standardowe klauzule umowne, Privacy Shield (jeśli powróci) itp. W polityce informujemy, że nie przekazujemy danych podmiotom trzecim bez

konieczności – wyjątkiem są np. integracje z Google/Twitch (ale tam user sam się loguje u nich).

Dzieci i dane wrażliwe: Platforma nie jest skierowana do dzieci poniżej 16 lat. Weryfikacja wieku odbywa się przez założenie konta w serwisie społecznościowym (np. Google wymaga 16+ w UE). Nie przetwarzamy danych szczególnych kategorii (typu poglądy polityczne, zdrowotne itp.) – a jeśli twórca coś takiego umieści w opisie, to jest to jego decyzja, nie my.

Zgodność finansowa i AML/KYC

Brak pełnego KYC na starcie: Ponieważ TipJar obsługuje mikropłatności, początkowo przyjmujemy, że większość transakcji pojedynczo i sumarycznie mieści się poniżej progów wymagających identyfikacji użytkowników (wg dyrektyw AML wiele jurysdykcji ma próg np. €1000 jednorazowo lub w miesiącu, powyżej którego wymagana jest weryfikacja). Twórcy, otrzymując drobne kwoty, nie muszą być od razu weryfikowani dokumentem. Podobnie fani płacąc drobne sumy.

KYC w razie potrzeby: TipJar jednak przygotowuje infrastrukturę na ewentualność wprowadzenia KYC:

Jeśli twórca chce wypłacić znaczną kwotę fiat na konto bankowe, może zostać poproszony o weryfikację tożsamości (podobnie jak np. PayPal to robi powyżej pewnych limitów).

Współpracując z Circle, musimy spełniać ich compliance – być może dla użycia Payouts API wymagane jest KYC twórcy jako tzw. sub-account. Wtedy TipJar musiałby zbierać od twórcy minimum danych (prawdziwe imię, nazwisko, kraj) do przekazania Circle do weryfikacji. Wdrożymy to tylko jeśli konieczne.

Fani płacący kartą – tutaj KYC zapewnia sam operator karty (bank). TipJar nie musi ich weryfikować, bo dostaje już środki od instytucji finansowej.

AML i przeciwdziałanie nadużyciom: System monitoruje nietypowe aktywności:

Np. jeden fan wysyła do wielu twórców po kolei maksymalne kwoty – może to być pranie pieniędzy lub oszustwo kartowe. TipJar może wtedy oznaczyć takie transakcje do kontroli.

W razie zgłoszenia organów ścigania, TipJar (mając minimalne dane, np. adresy portfeli) współpracuje – to w skrajnych przypadkach. Ważne, by w regulaminie zastrzec sobie prawo zamrożenia środków i kont, jeśli istnieje podejrzenie naruszenia prawa.

Sanctions screening: Ponieważ globalnie różnie bywa, TipJar powinien zapewnić, że nie obsługuje podmiotów objętych sankcjami (OFAC). Circle pewnie to robi po swojej stronie (jeśli np. ktoś z kraju z listy próbowałby).

Licencje i regulacje lokalne: TipJar operuje w obszarze, który może podpadać pod definicję usługi przekazu pieniężnego lub portfela elektronicznego. W wielu krajach to regulowane działalności (np. w USA Money Service Business wymaga licencji stanowych, w UE – licencja małej instytucji płatniczej powyżej pewnej skali). Strategią TipJar może być:

Na początek działać jako pośrednik techniczny korzystający z licencji partnera (tu Circle spełnia rolę regulated entity – wydawcy e-pieniądza USDC).

W miarę wzrostu, skonsultować się z prawnikami co do konieczności uzyskania własnych licencji. Np. w UE, jeśli TipJar ma siedzibę w kraju UE, a przechowuje środki użytkowników, być może musi zarejestrować się jako AIS/EMS (instytucja pieniądza elektronicznego) lub znaleźć umbrella arrangement.

Ustaliliśmy, że twórcy formalnie posiadają portfele custodial kontrolowane przez TipJar – to rodzi obowiązek odpowiedzialnego przechowywania środków. W regulaminie będzie zapis, że środki nie są ubezpieczone jak depozyty bankowe, ale trzymane 1:1 w rezerwie.

Podatki i rozliczenia: TipJar musi umożliwić twórcom dostęp do historii transakcji i sum, aby mogli się rozliczyć w swoim kraju (np. jako darowizny czy przychód). Być może w przyszłości TipJar zaoferuje generowanie raportów podatkowych (np. ile twórca zarobił w roku – ułatwienie dla niego).

Sam TipJar jako firma też odprowadza podatki od swoich prowizji, to już kwestia finansowa firmy.

Zgodność z ToS platform zewnętrznych: Jeśli twórca integruje TipJar z YouTube/Twitch (np. daje linki, używa powiadomień), trzeba dbać, by nie łamało to ich regulaminów (niektóre platformy mają własne systemy napiwków i nie lubią konkurencji, ale dopóki TipJar jest zewnętrznym linkiem, zwykle jest ok).

W przypadku Twitcha np. używanie zewnętrznych donosów jest powszechne (Streamlabs), więc tu nie będzie problemu. Dla YouTube – mają SuperChat, ale linki do zewnętrznych darowizn też ludzie dają, to raczej dozwolone.

Zgodność z przepisami dotyczącymi kryptowalut

Stablecoin USDC zgodny regulacyjnie: USDC jest wydawany przez regulowaną instytucję (Circle, pod nadzorem FinCEN w USA). Korzystanie z USDC daje pewność co do jego rezerwy i stabilności (mniejsze ryzyko niż np. nieaudytowane stablecoiny). TipJar nie emituje własnego tokena, tylko używa istniejącego – to upraszcza sprawy prawne (nie jest emitentem).

Smart kontrakty (jeśli wprowadzone): Na razie TipJar nie planuje własnych smart kontraktów (oprócz ewentualnego użycia gotowego Paymastera lub implementacji subskrypcji). Gdyby

jednak coś takiego było potrzebne, ich kod podlega audytowi bezpieczeństwa, a także powinien być licencjonowany i zgodny z prawnymi aspektami (np. jak kontrakty zbierają środki – by nie zostały uznane za nielegalną działalność inwestycyjną, trzeba jasno określić cel).

Polityka treści i prawa autorskie

Moderacja profili: Twórcy nie mogą umieszczać treści nielegalnych (mowa nienawiści, pornografia dziecięca etc.) na swoich profilach TipJar. W regulaminie jest to zabronione. TipJar będzie reagował na zgłoszenia takich treści (np. usuwał je, banował konta).

Prawa autorskie: Jeśli twórca użyje np. cudzego loga czy zastrzeżonej grafiki na swoim profilu, formalnie odpowiedzialność jest po jego stronie (platforma jako pośrednik hostingowy może stosować mechanizm notice-and-takedown). Warto mieć procedurę DMCA zgłaszania naruszeń praw autorskich.

Prywatność użytkowników: Oprócz danych osobowych, TipJar chroni też prywatność transakcji – tzn. domyślnie, jeśli fan chce zostać anonimowy, to jego tożsamość nie jest ujawniana twórcy ani innym. Nawet jak fan się zaloguje, może wybrać anonimowość per transakcja. TipJar oczywiście widzi powiązanie (bo w bazie ma userId fana), ale nie udostępnia go dalej.

Profilowanie i reklamy: TipJar nie planuje sprzedawać danych użytkowników czy wyświetlać reklam targetowanych. Model biznesowy opiera się na prowizjach, więc nie ma potrzeby naruszać prywatności dla reklam. To warto podkreślać jako zaletę (w odróżnieniu od np. platform społecznościowych).

Podsumowując, compliance dla TipJar to ciągły proces: od startu na uproszczonych zasadach (dzięki low-risk na mikropłatnościach) po ewentualne dostosowywanie się do wymogów w miarę rozwoju. Już na etapie projektowania jednak uwzględniamy te aspekty, by uniknąć kosztownych zmian w przyszłości (lepiej mieć moduł KYC gotowy, nawet jeśli wyłączony). Współpraca z doświadczonym doradcą prawnym od fintech/krypto jest przewidziana przed wejściem na rynek, aby upewnić się, że platforma spełnia wymogi w jurysdykcjach, w których będzie działać.

Key Code Snippets (NestJS & React)

W tej sekcji przedstawiamy kluczowe fragmenty kodu demonstrujące niektóre rozwiązania zaimplementowane w TipJar – zarówno po stronie backendu (NestJS), jak i frontendu (React). Są to uproszczone przykłady ilustrujące integrację z zewnętrznymi usługami i mechanizmami aplikacji.

Przykład (Backend NestJS): Integracja OAuth z Google oraz generowanie JWT

Poniższy fragment kodu w NestJS pokazuje implementację strategii logowania przez Google (OAuth 2.0) z wykorzystaniem Passport.js, a następnie użycie serwisu AuthService do zweryfikowania/utworzenia użytkownika oraz wygenerowania tokenu JWT. Składa się on z:

GoogleStrategy – klasa PassportStrategy definiująca, jak pobrać dane profilu Google i co z nimi zrobić.

AuthController – endpointy GET /auth/google (inicjujące przekierowanie) i /auth/google/callback (odbierające powrót z tokenem od Google).

```
// google.strategy.ts (NestJS)
@Injectable()
export class GoogleStrategy extends PassportStrategy(Strategy, 'google') {
  constructor(private authService: AuthService) {
    super({
      clientID: process.env.GOOGLE_CLIENT_ID, // zdefiniowane w .env
      clientSecret: process.env.GOOGLE_CLIENT_SECRET,
      callbackURL: process.env.GOOGLE_CALLBACK_URL, // np.
      "https://api.tipjar.com/auth/google/callback"
      scope: ['email', 'profile'],
    });
  }

  async validate(accessToken: string, refreshToken: string, profile: Profile, done:
  VerifyCallback): Promise<any> {
    // Pobieramy najważniejsze dane z profilu Google
    const { id: googleId, name, emails, photos } = profile;
    const email = emails?.[0]?.value;
    const displayName = name?.givenName ? `${name.givenName} ${name.familyName ||
    ""}`.trim() : profile.displayName;
    const avatarUrl = photos?.[0]?.value;

    if (!googleId || !email) {
      // Brak kluczowych danych – przerywamy z błędem
      return done(new HttpException('Brak wymaganych danych Google',
      HttpStatus.UNAUTHORIZED), false);
    }
    try {
      // Sprawdzamy lub tworzymy użytkownika na podstawie Google ID / email
      const user = await this.authService.validateOAuthUser('google', googleId, email,
      displayName, avatarUrl);
      // Kontynuujemy proces logowania – przekazujemy obiekt user dalej (będzie dostępny w
      req.user)
      done(null, user);
    } catch (err) {
      done(err, false);
    }
  }
}
```

```

}
}

// auth.controller.ts (NestJS)
@Controller('auth')
export class AuthController {
  constructor(private authService: AuthService) {}

  @Get('google')
  @UseGuards(AuthGuard('google'))
  googleAuth(@Req() req: Request) {
    // Ten endpoint służy tylko do przekierowania przez Guard -> PassportStrategy zajmie się
    // przekierowaniem.
  }

  @Get('google/callback')
  @UseGuards(AuthGuard('google'))
  async googleCallback(@Req() req: Request, @Res() res: Response) {
    // To miejsce trafia użytkownik po pomyślnym zalogowaniu przez Google (Validate zwrócił
    // obiekt user)
    const user = req.user as TipJarUser; // zakładamy, że AuthService zwraca obiekt typu
    // TipJarUser
    if (!user) {
      throw new HttpException('Uwierzytelnianie Google nie powiodło się',
        HttpStatus.UNAUTHORIZED);
    }
    // Generujemy JWT dla zalogowanego użytkownika
    const jwtToken = await this.authService.login(user);
    // Ustawiamy JWT w ciasteczku (HttpOnly) lub zwracamy w body – tutaj przykład cookie:
    res.cookie('jwt', jwtToken.accessToken, {
      httpOnly: true,
      secure: process.env.NODE_ENV === 'production',
      sameSite: 'lax',
      maxAge: 3600 * 1000 // 1h ważności
    });
    // Przekierowujemy użytkownika na frontend (np. pulpit twórcy)
    return res.redirect(`${process.env.FRONTEND_URL}/dashboard`);
  }
}

```

W powyższym kodzie warto zauważyć:

Użycie dekoratora `@UseGuards(AuthGuard('google'))` automatycznie przekierowuje niezalogowanego użytkownika do strony logowania Google przy wejściu na `/auth/google`, a następnie obsługuje powrót na `/auth/google/callback`.

`AuthService.validateOAuthUser(provider, providerId, email, name, avatar)` – ta metoda wewnątrz:

Sprawdza w bazie, czy istnieje już użytkownik z takim googleId lub e-mail (żeby nie tworzyć duplikatów).

Jeśli nie, tworzy nowy rekord User z tymi danymi (i np. generuje dla niego username na podstawie nazwy).

Jeżeli nowo utworzony, inicjuje stworzenie portfela Circle dla tego użytkownika (np. wysyłając zdarzenie do kolejki – dalej omówione).

Zwraca obiekt użytkownika (bezpieczny, np. bez haseł bo tu nie ma haseł).

Po udanej autoryzacji generujemy JWT (authService.login(user) może korzystać z biblioteki @nestjs/jwt do wystawienia tokenu podpisanego naszym sekretem, zawierającego np. sub: userId, role itp.). Ten token jest przekazywany frontendowi – w przykładzie poprzez cookie, co jest wygodne bo wtedy kolejne zapytania mogą go wysyłać automatycznie (cookie HttpOnly).

Ostatecznie następuje przekierowanie użytkownika na aplikację frontendową. Frontend po otrzymaniu tego żądania, odczyta cookie JWT i uzna użytkownika za zalogowanego.

Przykład (Backend NestJS): Tworzenie portfela Circle dla nowego użytkownika

Gdy twórca rejestruje się (np. przez Google jak wyżej), TipJar tworzy dla niego portfel USDC poprzez integrację z Circle Programmable Wallets API. Poniższy pseudo-kod pokazuje, jak mogłaby wyglądać metoda CircleService.provisionUserWallet wywoływana asynchronicznie po rejestracji:

```
// circle.service.ts (NestJS) – fragment ilustrujący tworzenie portfela Circle
@Injectable()
export class CircleService implements OnModuleInit {
  private circleClient: CircleSDK; // zakładamy istnienie klasy SDK od Circle

  onModuleInit() {
    this.circleClient = new CircleSDK({
      apiKey: process.env.CIRCLE_API_KEY,
      entityId: process.env.CIRCLE_ENTITY_ID, // identyfikator podmiotu (TipJar)
      // ... inne opcje, np. url sandbox/production
    });
  }

  async provisionUserWallet(userId: string, email: string): Promise<void> {
    const idempotencyKey = uuidv4();
    try {
      const response = await this.circleClient.wallets.createWallet({
        idempotencyKey,
```

```

walletSetId: process.env.CIRCLE_WALLET_SET_ID,    // zdefiniowany zestaw portfeli
description: `TipJar Wallet for user ${userId}`,
// Tworzymy jeden portfel na sieci (np. Polygon) typu SCA:
wallets: [{
  blockchain: process.env.DEFAULT_BLOCKCHAIN || 'POLYGON',
  chain: process.env.DEFAULT_BLOCKCHAIN_CHAIN || 'MATIC_MAINNET',
  // ^ przykładowe parametry zależne od SDK Circle
  accountType: 'SCA',
  metadata: { userId, email }
}]
});
const circleWallet = response.data.wallets[0];
// Zakładamy, że otrzymujemy obiekt portfela z polami walletId i addresses:
const circleWalletId = circleWallet.walletId;
const blockchainAddress = circleWallet.addresses?.[0]?.address;
// Zapisujemy te dane w naszej bazie (user.circleWalletId, user.mainWalletAddress)
await this.userService.update(userId, {
  circleWalletId,
  mainWalletAddress: blockchainAddress
});
this.logger.log(`Utworzono portfel Circle dla user ${userId}: ${circleWalletId}`);
} catch (error) {
  this.logger.error(`Błąd tworzenia portfela Circle dla ${userId}: ${error.message}`);
  throw new InternalServerErrorException('Nie udało się utworzyć portfela');
}
}
}

```

Kilka wyjaśnień:

CircleSDK.wallets.createWallet(...) – to przykładowe wywołanie; w rzeczywistości SDK Circle może mieć nieco inną składnię. Ważne jest przekazanie walletSetId (który konfiguruje się w systemie Circle – to grupuje portfele użytkowników TipJar) oraz parametrów portfela. Używamy accountType: 'SCA' dla opcji sponsorowania gazu.

Po otrzymaniu wyniku, pobieramy walletId (unikalny identyfikator w systemie Circle) i pierwszy adres blockchain (Circle może tworzyć adresy depozytowe).

Te informacje zapisujemy w bazie, by później móc np. wyświetlić adres portfela twórcy (gdy fan chce zapłacić krypto, ten adres jest docelowy), a także by móc wykonywać operacje (transfery) przez API Circle używając circleWalletId.

Obsługa błędów: W przypadku wyjątku rzucający błąd 500 – choć ta metoda i tak jest wywoływana w tle, więc ewentualnie logujemy i możemy spróbować ponownie (np. mechanizm kolejki z automatycznym retry).

Wołamy provisionUserWallet asynchronicznie, np. w AuthService:


```

// auth.service.ts (fragment)
async validateOAuthUser(provider: string, providerId: string, email: string, name: string,
avatar: string) {
  let user = await this.userRepository.findOne({ where: { [`${provider}Id`]: providerId } });
  if (!user) {
    // Jeśli nie znajdziemy po googleId/twitchId, sprawdzamy e-mail (możliwe, że istnieje
    // konto utworzone inną metodą z tym samym mailem)
    user = await this.userRepository.findOne({ where: { email } });
  }
  if (user) {
    // Jeśli jest, a nie miał przypisanego tego providerId, to przypisz (łączenie kont)
    if (!user[`${provider}Id`]) {
      user[`${provider}Id`] = providerId;
      await this.userRepository.save(user);
    }
  } else {
    // Tworzymy nowego użytkownika
    user = this.userRepository.create({
      email,
      displayName: name,
      avatarUrl: avatar,
      [`${provider}Id`]: providerId,
      username: this.generateUsernameFromEmail(email), // prosty alias
    });
    await this.userRepository.save(user);
    // >>> Uruchamiamy asynchronicznie tworzenie portfela (np. dodając do kolejki)
    this.eventEmitter.emit('user.created', { userId: user.id, email: user.email });
    // alternatywnie: await this.circleService.provisionUserWallet(user.id, user.email);
  }
  return user;
}

```

Tutaj po utworzeniu usera wywołujemy asynchronicznie `circleService.provisionUserWallet`. Można to zrobić eventem (jak wyżej), albo wrzucić zadanie do kolejki BullMQ, jeśli wolimy kolejki.

Przykład (Frontend React): Integracja z MetaMask (Sign-In with Ethereum)

Ten fragment kodu React (TypeScript, np. używając `ethers.js`) pokazuje uproszczony proces logowania portfelem Ethereum (SIWE). Użytkownik klika przycisk "Zaloguj portfelem", a aplikacja:

1. Łączy się z MetaMaskem i uzyskuje adres użytkownika.
2. Pobiera nonce z backendu (zapobiega replay attacks).

3. Tworzy wiadomość wg standardu EIP-4361 i prosi użytkownika o podpis.

4. Wysyła podpisaną wiadomość do backendu do weryfikacji i uzyskania JWT.

```
// LoginWithEthereum.tsx (React komponent)
```

```
import { providers } from 'ethers';
```

```
async function signInWithEthereum() {
```

```
  if (!window.ethereum) {
```

```
    alert('Zainstaluj MetaMask, aby kontynuować');
```

```
    return;
```

```
  }
```

```
  const provider = new providers.Web3Provider(window.ethereum);
```

```
  await provider.send('eth_requestAccounts', []); // popros o dostep do kont
```

```
  const signer = provider.getSigner();
```

```
  const address = await signer.getAddress();
```

```
  // 1. Pobierz nonce od backendu
```

```
  const nonceResponse = await fetch('/api/auth/siwe/nonce');
```

```
  const nonce = await nonceResponse.text();
```

```
  // 2. Zbuduj wiadomość SIWE
```

```
  const domain = window.location.host;
```

```
  const statement = 'Logowanie do TipJar'; // tekst wyświetlany w portfelu
```

```
  const uri = window.location.origin;
```

```
  const version = '1';
```

```
  const chainId = (await provider.getNetwork()).chainId;
```

```
  const message = `
```

```
service: ${domain}
```

```
domain: ${domain}
```

```
address: ${address}
```

```
statement: ${statement}
```

```
uri: ${uri}
```

```
version: ${version}
```

```
nonce: ${nonce}
```

```
issuedAt: ${new Date().toISOString()}
```

```
chainId: ${chainId}
```

```
`;
```

```
  // Uwaga: Powyższy format to uproszczenie; zgodnie z EIP-4361 powinna to być sformatowana treść.
```

```
  // 3. Poproś użytkownika o podpisanie wiadomości
```

```
  const signature = await signer.signMessage(message);
```

```
  // 4. Wyślij do backendu celem weryfikacji
```

```

const verifyResponse = await fetch('/api/auth/siwe/verify', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ message, signature })
});
if (verifyResponse.ok) {
  // Logowanie udane – backend ustawił cookie JWT lub zwrócił token
  window.location.reload(); // odśwież, by wczytać dane zalogowanego użytkownika
} else {
  alert('Logowanie portfelem nie powiodło się');
}
}

```

Wyjaśnienia do powyższego:

Sprawdzamy, czy `window.ethereum` jest dostępny (czyli czy MetaMask jest zainstalowany). Jeśli nie, informujemy użytkownika.

Używamy `Ethers.js providers.Web3Provider` do integracji z MetaMask. Wywołanie `eth_requestAccounts` wyświetla użytkownikowi dialog MetaMaska o zgodę na połączenie z witryną.

`getSigner().getAddress()` daje nam adres użytkownika – będzie potrzebny w wiadomości SIWE.

Zapytanie do `/api/auth/siwe/nonce` oczekuje, że backend wygenerował nonce (np. losowy string i trzyma go w pamięci lub w sesji). Nonce jest potrzebny, by podpis nie mógł być wykorzystany ponownie.

Tworzymy treść wiadomości. Uwaga: Standard EIP-4361 definiuje format bardzo konkretnie (z nagłówkiem `domain wants you to sign in...`). W produkcji należy użyć dedykowanych bibliotek generujących wiadomość SIWE (np. `siwe npm package`). Tutaj dla czytelności rozpisaliśmy elementy.

`signer.signMessage(message)` otwiera MetaMask z prośbą o podpisanie tekstu. Użytkownik widzi czytelną wiadomość (dlatego `statement` i `domain` są ważne).

Po uzyskaniu `signature`, wysyłamy ją na backend do weryfikacji. Backend (NestJS) w endpointzie `/auth/siwe/verify`:

Odbierze `message` i `signature`.

Samodzielnie odtworzy z `message` parametry (`nonce`, adres itp.) i za pomocą `ethers.utils.verifyMessage(message, signature)` odzyska adres podpisującego. Porówna go z adresem w treści wiadomości.

Sprawdzi, czy `nonce` się zgadza z tym, co wygenerował i czy jeszcze nie był użyty (żeby ktoś nie przesłał dwa razy tego samego).

Jeśli wszystko OK, to albo znajdzie użytkownika z tym adresem, albo go utworzy (jak wcześniej w AuthService – np. walletAddressSIWE pola użyć).

Wygeneruje JWT dla tego użytkownika i zwróci sukces (tu zakładamy, że stawia HttpOnly cookie).

Po stronie frontendu, jeśli verifyResponse.ok (HTTP 200), to znaczy że zalogowano. Możemy np. przeładować stronę, aby UI zaktualizował się na zalogowany stan. Można też od razu wywołać jakiś kontekst uwierzytelnienia.

Przykład (Backend NestJS): Wywołanie transferu USDC poprzez Circle API

Założmy scenariusz, że fan posiada portfel w TipJar (Circle) z pewnym saldem USDC i chce przesłać 10 USDC do twórcy (wewnętrzny transfer). Pokazujemy uproszczony kod akcji backendu inicjującej transfer za pomocą Circle Transfers API:

```
// payments.service.ts (NestJS) – fragment obsługi transferu USDC w ramach Circle
async tipViaInternalTransfer(fanId: string, creatorId: string, amount: number) {
  // Pobierz dane portfeli z bazy
  const fan = await this.userRepository.findOne(fanId);
  const creator = await this.userRepository.findOne(creatorId);
  if (!fan || !creator) throw new NotFoundException('User not found');
  if (!fan.circleWalletId || !creator.circleWalletId) {
    throw new BadRequestException('One of users does not have a Circle wallet');
  }
  // Sprawdź saldo fana (np. przechowywane lub poprzez API Circle - tutaj zakładamy, że w
  // bazie lub cache mamy aktualne saldo)
  if (fan.balance < amount) {
    throw new BadRequestException('Insufficient balance');
  }
  // Przygotuj dane do transferu
  const idemKey = uuidv4();
  try {
    const transferRes = await this.circleService.transferUSDC(fan.circleWalletId,
    creator.circleWalletId, amount, idemKey);
    // Założmy, że transferUSDC zwraca jakiś obiekt z danymi transferu
    this.logger.log(`Transfer initiated: ${fan.id} -> ${creator.id}, amount: ${amount} USDC`);
    // Zapisz w bazie nowy Tip
    await this.tipRepository.save({
      fromUserId: fan.id,
      toUserId: creator.id,
      amount,
      status: 'CONFIRMED', // transfer wewn. zakładamy natychmiastowy
      method: 'USDC_INTERNAL',
      timestamp: new Date()
    });
  } catch (error) {
    // ...
  }
}
```

```

});
// Zaktualizuj salda lokalnie
fan.balance -= amount;
creator.balance += amount;
await this.userRepository.save([fan, creator]);
return transferRes;
} catch (error) {
  this.logger.error(`USDC internal transfer failed: ${error.response?.data || error.message}`);
  throw new HttpException('Transfer nieudany', HttpStatus.BAD_GATEWAY);
}
}

```

I odpowiednia metoda w CircleService:

```

// circle.service.ts
async transferUSDC(fromWalletId: string, toWalletId: string, amount: number,
idempotencyKey: string) {
  // Kwota w minimalnych jednostkach? Circle może wymagać np. najpierw stringa lub
  struktury { amount: '10.00', currency: 'USD' }
  const transferBody = {
    idempotencyKey,
    source: { type: 'wallet', id: fromWalletId },
    destination: { type: 'wallet', id: toWalletId },
    amount: { amount: amount.toFixed(2), currency: 'USD' } // USDC ma 2 miejsca po
    przecinku jak USD
  };
  return await this.circleClient.transfers.createTransfer(transferBody);
}

```

Ten kod obrazuje:

Sprawdzenie pre-warunków (istnienie użytkowników, posiadanie portfeli, wystarczające saldo).

Wywołanie `circleClient.transfers.createTransfer` – ta metoda (dostarczona przez SDK Circle) wykonuje HTTP POST do `/v1/transfers`. Circle następnie realizuje transfer wewnętrznie. Często taki transfer jest natychmiastowy, ale dla pewności najlepiej sprawdzić status w odpowiedzi (powinien być `complete`).

Zapis transakcji w naszej bazie Tip i aktualizacja sald (jeśli prowadzimy lokalny stan sald).

Obsługa błędów z odpowiednim logowaniem.

W `error.response?.data` może być informacja od API Circle (np. „insufficient funds” – co raczej wykryliśmy wcześniej, lub „wallet not found”).

Zwracamy błąd 502 (Bad Gateway) lub inny, by frontend wiedział, że coś poszło nie tak z zewnętrznym serwisem.

Przykład (Frontend React): Formularz wysłania napiwku z użyciem API backendu

Na koniec pokazujemy prosty fragment interfejsu React odpowiedzialnego za wysłanie napiwku poprzez TipJar (np. fan zalogowany, korzysta z salda TipJar – scenariusz wewnętrznego transferu):

```
// TipForm.jsx (React)
import { useState } from 'react';

function TipForm({ creatorId }) {
  const [amount, setAmount] = useState(5);
  const [message, setMessage] = useState("");
  const [status, setStatus] = useState(null);

  const submitTip = async (e) => {
    e.preventDefault();
    setStatus('pending');
    try {
      const res = await fetch('/api/tips', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ creatorId, amount, message })
      });
      if (!res.ok) throw new Error('Server error');
      setStatus('success');
    } catch (err) {
      console.error('Tip failed:', err);
      setStatus('error');
    }
  };

  if (status === 'success') {
    return <div className="tip-success">Dziękujemy za napiwek!</div>;
  }

  return (
    <form onSubmit={submitTip}>
      <label>
        Kwota (USDC):
        <input type="number" min="1" max="1000" value={amount} onChange={e =>
setAmount(+e.target.value)} />
      </label>
      <label>
        Wiadomość (opcjonalnie):

```

```

        <input type="text" maxLength="100" value={message} onChange={e =>
setMessage(e.target.value)} />
    </label>
    <button type="submit">Wesprzyj twórcę</button>
    {status === 'pending' && <div className="loading">Przetwarzanie...</div>}
    {status === 'error' && <div className="error">Nie udało się wysłać wsparcia. Spróbuj
ponownie.</div>}
    </form>
  );
}

```

Co tu się dzieje:

Komponent przyjmuje creatorId (identyfikator twórcy, któremu wysyłamy napiwek – uzyskany np. z URL strony lub z kontekstu).

Lokalny stan amount i message trzyma wpisane wartości.

Po submit, wykonujemy `fetch('/api/tips', { method: 'POST', body: {...} })`.

Zakładamy, że użytkownik (fan) jest zalogowany, więc przeglądarka wyśle JWT w cookie albo my dołączymy nagłówek Authorization – to jest szczegół autoryzacji. Jeśli fan jest gościem, to ten endpoint mógłby wymagać np. podania też danych płatności – w tym scenariuszu uproszczonym skupiamy się na zalogowanym fanie.

Backend endpoint `/api/tips` zidentyfikuje użytkownika z JWT, wyciągnie creatorId z body i kwotę. Dalej wywoła `paymentsService.tipViaInternalTransfer(fanId, creatorId, amount)` jak pokazano wcześniej.

W UI pokazujemy stan oczekiwania („Przetwarzanie...”) i sukces lub błąd odpowiednio po zakończeniu fetch.

W realnej aplikacji moglibyśmy w przypadku sukcesu np. wyczyścić formularz i zaktualizować listę ostatnich napiwków od razu (optimistic update) albo przekierować gdzieś.

Ten fragment nie dotyczy bezpośrednio blockchain czy Circle, ale pokazuje prostotę użycia API przez frontend. Dla fanów niezaznajomionych z krypto, ważne jest, że interfejs jest zwykły – wpisują kwotę, klikają, dostają potwierdzenie – a cała złożoność (transakcje USDC, sponsorowanie gazu) dzieje się „pod maską” w warstwie backend.

Powyższe Key Code Snippets obrazują tylko wycinek implementacji TipJar, ale pokazują podejście do:

Integracji z zewnętrznymi usługami poprzez dedykowane moduły (Auth, CircleService).

Uwierzelniania użytkowników różnymi metodami (OAuth, SIWE).

Bezpiecznego wydawania tokenów sesyjnych (JWT) i przechowywania ich w cookie HttpOnly.

Korzystania z API Circle do obsługi portfeli i transferów USDC.

Interakcji frontendu z portfelami Web3 (podpisywanie wiadomości, wysyłanie transakcji on-chain).

Ogólnej komunikacji frontend-backend (fetch API).

Kod w docelowej bazie będzie oczywiście bardziej rozbudowany (obsługa więcej przypadków brzegowych, walidacje inputów, etc.), ale wyżej przedstawione fragmenty stanowią podstawę, na której budowana jest cała logika TipJar.

Podsumowanie

Dokument ten przedstawił pełne spojrzenie na projekt TipJar – od koncepcji, poprzez szczegółową specyfikację funkcjonalną, po techniczną architekturę systemu i jego implementację. TipJar ma potencjał stać się przełomowym narzędziem dla twórców treści, łącząc świat mikropłatności z nowoczesną technologią blockchain w sposób niewidoczny dla końcowego użytkownika (ukryta złożoność, przyjazny UX).

Kluczowe wyróżniki to wykorzystanie stablecoina USDC i infrastruktury Circle, co zapewnia stabilność wartości, globalny zasięg oraz eliminację barier technicznych (custodial wallets, gasless transactions). Dzięki temu twórcy mogą skupić się na swojej pracy, a fani – na okazywaniu wsparcia, nie zaś na zmaganiu się z przeszkodami płatniczymi.

Z punktu widzenia technologii, zespół deweloperski ma przed sobą wyzwanie integracji wielu elementów (backend, blockchain, płatności fiat, frontendy różnego rodzaju), ale dzięki przedstawionej architekturze modułowej i użyciu sprawdzonych narzędzi (NestJS, React, Docker, CI/CD) projekt jest wykonalny i skalowalny. Diagram architektury oraz fragmenty kodu zawarte w dokumencie mają służyć jako pomoc w zrozumieniu przepływów i stanowić punkt wyjścia przy implementacji.

Kolejnymi krokami zespołu będzie realizacja MVP zgodnie z wytycznymi tu zawartymi, etapowe testy i wdrożenie – najpierw w środowisku testowym, następnie publicznie. Równolegle należy dopracować kwestie prawne i regulacyjne, by zapewnić pełną zgodność działania platformy.

TipJar jest przedsięwzięciem łączącym innowację technologiczną z realną potrzebą rynkową, i przy odpowiedniej realizacji może z powodzeniem zająć niszę na rynku monetyzacji treści, przynosząc korzyści zarówno twórcom, jak i ich społecznościom fanów.

Zespół deweloperski, uzbrojony w niniejszą dokumentację, ma klarowną mapę drogową, by doprowadzić projekt od fazy koncepcji do w pełni funkcjonalnego produktu. Powodzenia w implementacji!